

Title: Time-Aware Sequential Neural Networks for Next-Best Product Recommendation

Name: Janaki Samba Moorthy

Submission Date: May 3rd, 2026

Note

The project was implemented across multiple Jupyter notebooks instead of a single notebook to keep each phase modular, readable, and easier to debug. Since the workflow includes data understanding, preprocessing, sequence construction, multiple model experiments, and final result consolidation, separating the work into phase-wise notebooks helped organize the project clearly and avoid making one very large, difficult-to-run file.

Text

Notebook	One-Line Description	Why It Was Included
NN Proj Phase 1 and 2 Janaki Samba Moorthy.ipynb	Performs initial data loading, data understanding, exploratory data analysis, and early preprocessing.	This notebook establishes the dataset foundation and shows key EDA findings before model preparation.
NN Proj Phase 3 Janaki Samba Moorthy.ipynb	Creates user-level chronological purchase sequences and prepares model-ready batch files.	This notebook is included because sequence construction and batch generation are the core preprocessing steps required for neural recommendation models.
NN Proj Phase4 LSTM+Relu Janaki Samba Moorthy.ipynb	Trains and evaluates the LSTM + ReLU + Attention model.	This notebook represents the main recurrent attention model and produced the best final Top-K recommendation performance.
NN Proj Phase 4 GRU+Swish Janaki Samba Moorthy.ipynb	Trains and evaluates the GRU + ReLU-SwiLU hybrid activation model.	This notebook tests whether a smoother SwiLU-based activation variant improves GRU recommendation performance.
NN Proj Phase 4 GRU+Relu+Gelu Janaki Samba Moorthy.ipynb	Trains and evaluates a GRU-based hybrid activation model variant.	This notebook is included as part of the activation-function comparison experiments.
NN Proj Phase 4 LSTM+Relu+Gelu Janaki Samba Moorthy.ipynb	Trains and evaluates the LSTM + ReLU-GELU hybrid activation model.	This notebook tests whether GELU activation improves LSTM-based sequence recommendation compared with standard ReLU.
NN Proj Phase 4 Tranformer Model Janaki Samba Moorthy.ipynb	Trains and evaluates the Transformer Encoder + GELU model.	This notebook provides a self-attention-based comparison against recurrent LSTM and GRU architectures.
NN Proj Phase 5 Consolidated Results Janaki Samba Moorthy.ipynb	Combines saved model outputs, creates final comparison tables, plots, and report-ready summaries.	This notebook is included to consolidate all model results in one place and support final interpretation.

Abstract

Next-best product recommendation is an important problem in modern e-commerce and online grocery platforms, where customer preferences are dynamic, repetitive, and strongly influenced by previous purchase behavior. Traditional recommendation methods such as popularity-based ranking, collaborative filtering, and content-based filtering are useful, but they often do not fully capture the sequential and time-dependent nature of shopping behavior. In grocery shopping, a customer's next purchase may depend on recently purchased products, product category relationships, time gaps between orders, day of week, hour of purchase, and personal shopping routines.

This project develops a **time-aware sequential neural network recommendation system** using the Instacart Market Basket dataset. The main objective is to predict the next product a customer is likely to purchase based on their historical purchase sequence and temporal shopping features. The task is formulated as a **multi-class classification problem**, where each product in the vocabulary represents a possible output class. The raw transaction data is transformed into user-level chronological sequences containing product IDs, aisle IDs, department IDs, day-of-week information, hour-of-day information, and recency features based on days since prior order.

The project follows an end-to-end deep learning workflow consisting of data understanding, exploratory data analysis, preprocessing, feature engineering, sequence construction, batch generation, model development, training, validation, and final evaluation. The implemented models include **LSTM + ReLU + Attention**, **GRU + ReLU + Attention**, **GRU + ReLU-SwiLU**, **LSTM + ReLU-GELU**, **Transformer Encoder + GELU**, and a **popularity-based baseline**. The recurrent models use embedding layers and attention mechanisms to learn sequential customer purchase behavior, while the Transformer Encoder model uses self-attention to model relationships across sequence positions. The final comparison notebook consolidates saved model outputs from CSV and JSON files, standardizes model names, removes duplicate rows, and generates final comparison tables and plots for Recall@10, Precision@10, and training performance.

The models are evaluated using both classification metrics and recommendation-focused Top-K metrics. Classification metrics include accuracy, weighted precision, weighted recall, and weighted F1-score. Recommendation metrics include Precision@5, Precision@10, Recall@5, Recall@10, Hit Rate@5, and Hit Rate@10. Since real-world recommendation systems usually return a ranked list of products rather than a single prediction, the Top-K metrics provide a more practical measure of recommendation usefulness.

Experimental results show that the **LSTM + ReLU + Attention** model achieved the strongest overall recommendation performance. It obtained the highest Recall@10 value of **0.146892**, meaning that the actual next product appeared in the model's top 10 recommendations for approximately **14.69%** of the test cases. The **GRU + ReLU + Attention** model performed very closely, achieving Recall@10 of **0.145853**. The hybrid activation models, including **GRU + ReLU-SwiLU** and **LSTM + ReLU-GELU**, produced valid comparison results but did not outperform the standard ReLU-based recurrent models. The **Transformer Encoder + GELU** model performed slightly lower than the recurrent models under the available computational setup.

Overall, the results demonstrate that time-aware recurrent neural networks are effective for modeling customer purchase behavior and generating personalized next-best product recommendations. The final findings show that recurrent attention-based architectures, especially **LSTM + ReLU + Attention** and **GRU + ReLU + Attention**, are better suited for this Instacart next-product recommendation task than the tested hybrid activation and lightweight Transformer variants. This project successfully provides a complete deep learning recommendation pipeline, from raw transaction data processing to model training, evaluation, final comparison, and report-ready result visualization.

Introduction

Problem Definition

This project addresses the problem of **next-best product recommendation** using time-aware sequential neural networks. In online grocery platforms, customers repeatedly purchase products over time, and their future purchases are often influenced by what they bought previously, how recently they bought it, and when they usually shop. Therefore, the recommendation task is naturally sequential and time-dependent.

The goal of this project is to predict the next product a customer is likely to purchase based on their past purchase history. Instead of treating purchases as independent events, each user's behavior is modeled as an ordered sequence. This allows the model to learn customer-specific shopping patterns and generate more personalized recommendations.

The prediction task is formulated as a **multi-class classification problem**, where each product in the product vocabulary is treated as a possible class. Given a customer's historical purchase sequence, the model predicts the next product.

Mathematically, a customer purchase sequence can be represented as: $X = [x_1, x_2, x_3, \dots, x_t]$ Where, each x_t represents a purchase event at time step t . The objective is to predict: $y = x_{t+1}$ where, y is the next product purchased by the customer. Each purchase event is represented using product, category, and temporal features:

$$x_t = [product_t, aisle_t, department_t, day_t, hour_t, recency_t]$$

where,

$product_t$ = encoded product ID
 $aisle_t$ = encoded aisle ID
 $department_t$ = encoded department ID
 day_t = order day of week
 $hour_t$ = order hour of day
 $recency_t$ = days since prior order

The neural model learns a nonlinear mapping:

$$\hat{y} = f_{\theta}(X)$$

Where,

f_{θ} = neural recommendation model
 X = user purchase history sequence
 $\hat{y}$ = predicted probability distribution over products
 θ = trainable model parameters

The main challenge in this task is that the product vocabulary is large, customer behavior is diverse, and grocery baskets often contain multiple reasonable future products. Therefore, exact next-product prediction is difficult. For this reason, the project evaluates models using both exact classification metrics and Top-K recommendation metrics.

Motivation and Real-World Relevance

Recommendation systems are widely used in e-commerce to help customers find products quickly and to improve user experience. In online grocery shopping, recommendation systems are especially important because customers often repeat purchases, buy related products together, and follow time-based shopping habits. A customer may buy milk every few days, fruits every week, or baking ingredients during specific occasions.

The motivation behind this project is to build a model that learns not only **what** a customer buys, but also **when** and **in what order** they buy products. Traditional popularity-based recommendation may suggest common products to everyone, but it does not personalize recommendations. Collaborative filtering can capture user-item similarities, but it does not naturally model the order of purchases. Content-based filtering can use product information, but it may miss evolving customer intent.

Sequential neural networks provide a more flexible solution because they learn directly from ordered customer behavior. By incorporating product features, category features, and temporal features, the model can generate recommendations that are more personalized and context-aware.

The real-world relevance of this project lies in its ability to connect deep learning-based recommendation with practical e-commerce and grocery retail applications. In online grocery platforms, customer purchases are often repetitive, time-sensitive, and influenced by previous shopping behavior. A time-aware sequential recommendation model can therefore help businesses understand what customers are likely to need next and generate product suggestions that are more personalized and timely.

Personalized product recommendation is one of the most direct applications of this project. Instead of recommending the same popular products to all customers, the proposed model learns from each user's historical purchase sequence and recommends products based on individual shopping behavior. This allows the system to suggest items that match a customer's preferences, purchase frequency, and recent activity. Such recommendations can be displayed on the homepage, product detail page, search page, cart page, or checkout page.

The model can also support basket completion and cross-selling. Grocery baskets often contain related products, and customers may forget items that commonly go together. A time-aware recommendation system can identify products that are likely to follow from the current or recent purchase sequence. For example, a customer buying pasta may be recommended pasta sauce, cheese, or related ingredients. This improves customer convenience while also increasing basket size and potential revenue.

Customer retention and engagement can be improved through accurate next-product recommendations. When customers receive useful and timely suggestions, they are more likely to return to the platform and complete repeat purchases. Since grocery shopping often involves recurring needs, a time-aware model can learn purchase cycles and recommend products when they are likely to be needed again. This helps create a more convenient shopping experience and strengthens long-term customer engagement.

The model can also support targeted promotions and coupon personalization. Instead of sending the same promotion to all customers, retailers can use predicted next-product preferences to deliver more relevant offers. For example, customers who frequently purchase baby products may receive coupons for diapers or wipes, while customers who often buy baking ingredients may receive discounts on flour or sugar. This improves promotion relevance and increases the likelihood of customer response.

Although the primary goal of the model is product recommendation, aggregated predictions can also support inventory and demand planning. Retailers can use recommendation trends to estimate which products customers may purchase soon, helping them prepare inventory, reduce stockouts, and avoid overstocking. This is especially useful in grocery retail, where product availability, freshness, and demand timing are critical.

Finally, the project supports retail customer behavior analysis. By analyzing purchase sequences, product-category relationships, recency patterns, and shopping time features, retailers can better understand how customers make purchasing decisions. These insights can reveal frequently repeated products, commonly co-purchased categories, and time-based shopping patterns. Overall, this project demonstrates how deep learning-based sequential recommendation can improve personalization, customer experience, marketing effectiveness, and retail decision-making.

Background and Related Work

Recommendation systems have evolved from simple ranking methods to deep learning-based personalization models.

- *Collaborative Filtering:* Collaborative filtering recommends products based on similarities among users or items. User-based collaborative filtering identifies users with similar behavior, while item-based collaborative filtering recommends products similar to items previously purchased. Matrix factorization methods improve this by learning hidden user and product embeddings. However, collaborative filtering can struggle with sparse data, cold-start users, and temporal behavior.
- *Sequential Recommendation:* Sequential recommendation models predict future user behavior based on ordered interaction histories. Recurrent Neural Networks such as LSTM and GRU are widely used for sequence modeling because they can process ordered events and learn dependencies across time.
- *Attention-Based Recommendation:* Attention mechanisms improve sequence modeling by allowing the model to focus on the most relevant past purchases. Instead of treating all

previous products equally, attention learns which sequence positions are most useful for predicting the next product.

- *Transformer-Based Recommendation:* Transformer models use self-attention to learn relationships across all positions in a sequence. Unlike LSTM and GRU models, they do not process data one step at a time. In this project, a lightweight Transformer Encoder with GELU activation is implemented for comparison against recurrent models. The Transformer code includes GELU activation, label smoothing, logit clamping, and NaN-safe handling.
- *Popularity-Based Recommendation:* Popularity-based recommenders rank products based on how frequently they are purchased. This method is simple and scalable, but it recommends the same products to every customer and does not personalize results.

Objectives and Contributions

The primary objective of this project is to develop a **time-aware sequential recommendation system** for next-product prediction using deep learning. The project models customer purchase histories as ordered sequences instead of independent transactions. This allows the system to learn customer preferences, product relationships, and time-based shopping behavior from historical grocery orders.

- *Data Understanding and Exploratory Data Analysis:* This objective focuses on understanding the structure and behavior of the Instacart Market Basket dataset before model development. The analysis includes loading the raw CSV files, checking dataset shapes, reviewing data types, identifying missing values, and studying shopping patterns such as orders per user, most purchased products, department-level trends, reorder behavior, order day patterns, order hour patterns, and days between purchases. This step confirms that grocery shopping behavior is sequential and time-dependent, making the dataset suitable for next-product recommendation.
- *User-Level Purchase Sequence Construction:* This objective focuses on converting raw transaction records into user-level chronological purchase sequences. Since the Instacart data is stored across multiple relational tables, order records are merged with product purchase records and product metadata. Each user's purchases are sorted by order number so that the natural order of shopping behavior is preserved. These ordered histories are then transformed into supervised learning samples, where previous purchases become the input sequence and the next product becomes the target label.
- *Product, Category, and Temporal Feature Engineering:* This objective focuses on creating meaningful features for each purchase event. Product IDs are used to capture exact item-level behavior, while aisle IDs and department IDs represent product hierarchy. Temporal features such as day of week, hour of day, and days since prior order

capture shopping timing and recency. Combining product, category, and temporal features gives the model a richer understanding of customer behavior than using product IDs alone.

- *Deep Learning Models for Next-Product Prediction:* This objective focuses on building neural network models that predict the next product a customer is likely to purchase. The task is formulated as a multi-class classification problem, where each product in the vocabulary is a possible output class. LSTM and GRU architectures are used because they are suitable for sequential data and can learn dependencies across customer purchase histories. Attention mechanisms are added to help the model focus on the most relevant previous purchases.
- *Model Architecture Comparison:* This objective focuses on comparing multiple recommendation models under the same experimental pipeline. The project compares a popularity baseline, LSTM with attention, GRU with attention, hybrid activation models, and a Transformer Encoder model. The popularity baseline provides a simple non-personalized benchmark, while the recurrent and Transformer-based models test different deep learning approaches for sequence modeling. This comparison helps identify which architecture is most suitable for next-product recommendation.
- *Classification and Top-K Evaluation:* This objective focuses on evaluating model performance using both classification metrics and recommendation-specific Top-K metrics. Accuracy, weighted precision, weighted recall, and weighted F1-score measure exact next-product prediction performance. Precision@5, Precision@10, Recall@5, Recall@10, Hit Rate@5, and Hit Rate@10 evaluate whether the true next product appears within the top recommended items. This is important because real-world recommendation systems usually show users a ranked list of products rather than only one prediction.
- *Best Model Identification and Practical Relevance:* This objective focuses on identifying the best-performing model and explaining its practical relevance. The final comparison shows that recurrent attention-based models performed strongest, with LSTM + ReLU + Attention achieving the best Top-K performance and GRU + ReLU + Attention performing very closely. This indicates that recurrent neural networks with attention are effective for learning grocery purchase sequences and can support practical applications such as personalized product recommendation, basket completion, cross-selling, customer engagement, and retail decision-making.

Major Technical Contributions

- *End-to-End Recommendation Pipeline:* A major contribution of this project is the development of a complete end-to-end recommendation pipeline. The workflow begins with raw Instacart transaction data and continues through data understanding, cleaning,

feature engineering, sequence construction, batch generation, model training, evaluation, and final comparison. This demonstrates how raw grocery transaction records can be transformed into a deep learning-based recommendation system.

- *Time-Aware Sequence Construction:* The project contributes a time-aware sequence construction process that preserves the chronological order of each customer's purchases. Instead of treating transactions as independent events, the system models them as ordered histories. Temporal features such as day of week, hour of day, and days since prior order are included to capture shopping routines, purchase gaps, and recency effects. This helps the model learn not only what customers buy, but also when they are likely to buy it.
- *Product Hierarchy Integration:* Another contribution is the integration of product hierarchy information into the recommendation model. In addition to product IDs, the model uses aisle IDs and department IDs as categorical inputs. This allows the system to learn relationships among similar products and broader shopping categories. Product hierarchy integration is useful because customers may show preferences for both specific items and larger categories such as produce, dairy, snacks, or beverages.
- *LSTM and GRU Attention-Based Models:* The project implements LSTM and GRU attention-based models for next-product prediction. LSTM models are useful for capturing longer-term dependencies in purchase histories, while GRU models provide a simpler and computationally efficient alternative. Attention mechanisms allow the models to focus on the most relevant previous purchases rather than treating all past items equally. This design is well suited for grocery recommendation because the next purchase may depend strongly on recent or related products.
- *Hybrid Activation Experiments:* The project explores hybrid activation experiments using GRU + ReLU-SwiLU and LSTM + ReLU-GELU variants. ReLU is simple and efficient, while SwiLU and GELU provide smoother nonlinear activation behavior. These experiments test whether alternative activation combinations can improve recommendation performance. Although the hybrid models produced valid results, they did not outperform the standard ReLU-based recurrent attention models in the final comparison.
- *Transformer Encoder Comparison:* The project includes a Transformer Encoder + GELU model as a self-attention-based comparison. Transformer models can learn relationships across all sequence positions using self-attention, unlike recurrent models that process sequences step by step. Including this model strengthens the experiment by allowing a comparison between recurrent sequence modeling and Transformer-style sequence modeling. In the final results, the Transformer model performed slightly lower than the recurrent attention models under the available computational setup.
- *Popularity Baseline Comparison:* The project includes a popularity baseline as a non-personalized benchmark. This baseline recommends the most frequently purchased products to every customer. It is important because grocery datasets often contain highly

repeated products, so a simple popularity-based model can still perform reasonably. Comparing deep learning models against this baseline helps determine whether time-aware sequential modeling provides meaningful improvement beyond recommending common products.

- *Classification and Top-K Recommendation Evaluation:* Another technical contribution is the use of both classification and recommendation-focused evaluation metrics. Accuracy, weighted precision, weighted recall, and weighted F1-score are used to evaluate exact next-product prediction. Precision@5, Precision@10, Recall@5, Recall@10, Hit Rate@5, and Hit Rate@10 are used to evaluate ranked recommendation performance. This dual evaluation approach gives a more realistic assessment of model usefulness in a real recommendation system.
- *Final Consolidated Model Comparison:* The final contribution is the creation of a consolidated model comparison. Results from saved CSV and JSON files are combined, cleaned, standardized, and organized into a final comparison table. This makes it possible to compare all models using the same metrics and clearly identify the best-performing architecture. The final comparison shows that LSTM + ReLU + Attention achieved the strongest Top-K recommendation performance, followed closely by GRU + ReLU + Attention.

Novel Aspects of the Project

- *Time-Aware Sequential Recommendation Architecture:* A key novel aspect of this project is the use of a time-aware sequential architecture for next-product prediction. Instead of treating customer purchases as independent records, the model models each user's behavior as an ordered sequence of purchases. This allows the system to learn how previous purchases influence future purchases. Temporal features such as day of week, hour of day, and days since prior order are included to capture real grocery shopping patterns such as weekly routines, repeat purchases, and recency-based behavior.
- *Multi-Input Feature Representation:* The model uses a richer input design than a product-only recommender. Each purchase event is represented using product ID, aisle ID, department ID, day-of-week value, hour-of-day value, and days-since-prior-order value. This creates a unified representation that combines item-level behavior, product hierarchy, and temporal context. By using product-, category-, and time-based features together, the model can learn more meaningful patterns of customer preferences.
- *Product Hierarchy Integration Using Aisle and Department Features:* Another novel aspect is the integration of Instacart's product hierarchy into the recommendation model. Instead of only predicting based on exact product IDs, the model also uses aisle and department information. This helps the neural network understand broader category relationships among products. For example, even if two products are different, they may

belong to the same aisle or department, allowing the model to learn category-level shopping behavior.

- *Recurrent Attention-Based Recommendation Models:* The project implements LSTM and GRU models with attention mechanisms. LSTM and GRU networks are suitable for sequence modeling because they can learn dependencies across previous purchases. The attention layer adds another improvement by helping the model focus on the most relevant parts of the user's purchase history. This is useful because not every previous purchase contributes equally to the next product prediction.
- *Hybrid Activation Function Experiments:* The project explores hybrid activation-based model variants, including GRU + ReLU-SwiLU and LSTM + ReLU-GELU. ReLU is simple and efficient, while SwiLU and GELU provide smoother nonlinear transformations. These experiments test whether combining activation behavior can improve recommendation learning. Although the hybrid activation models produced valid results, the final comparison showed that they did not outperform the standard ReLU-based recurrent attention models.
- *Transformer Encoder Comparison:* The project also includes a Transformer Encoder + GELU model as a self-attention-based comparison. This strengthens the study by comparing traditional recurrent sequence models with a Transformer-style architecture. The Transformer Encoder uses self-attention to learn relationships across sequence positions, while LSTM and GRU models process sequences recurrently. In the final results, the Transformer model performed slightly lower than the recurrent attention models under the available computational setup.
- *Popularity Baseline for Practical Benchmarking:* A popularity-based baseline is included to provide a simple non-personalized benchmark. This is important because grocery datasets often contain frequently purchased products, and a simple popularity model can still perform reasonably well. By comparing neural models against this baseline, the project evaluates whether time-aware sequential learning provides practical value beyond recommending the most common products.
- *Recommendation-Oriented Top-K Evaluation:* The project goes beyond standard classification metrics by using Top-K recommendation metrics such as Precision@5, Precision@10, Recall@5, Recall@10, Hit Rate@5, and Hit Rate@10. This is important because real-world recommendation systems usually show customers a ranked list of suggested products rather than only one predicted product. Therefore, Top-K evaluation provides a more realistic measure of recommendation usefulness.
- *Scalable Batch-Based Training Pipeline:* The dataset is large, so the project uses a batch-based training pipeline to make model training feasible in Google Colab. Processed user sequences are saved into batch files and loaded during training and evaluation. This improves scalability and avoids memory issues when working with millions of

transaction records. The final output notebook also consolidates saved CSV and JSON result files to create report-ready comparison tables and plots

Summary of Technical Contributions

- *Complete End-to-End Recommendation Pipeline:* This project developed a complete end-to-end recommendation pipeline using the Instacart Market Basket dataset. The workflow includes data understanding, exploratory data analysis, preprocessing, feature engineering, user-level sequence construction, batch generation, model training, evaluation, and final comparison. This makes the project a full recommendation system workflow rather than only a model implementation.
- *Time-Aware User Sequence Construction:* The project transformed raw transaction data into user-level chronological purchase sequences. Each user's previous purchases were used as input, and the next product was used as the target label. The use of chronological ordering allows the model to learn sequential customer behavior and changing purchase intent over time.
- *Product, Category, and Temporal Feature Engineering:* A major technical contribution is the engineering of multiple feature groups. The project uses product IDs, aisle IDs, department IDs, day-of-week values, hour-of-day values, and days-since-prior-order values. This feature design allows the model to learn from exact products, product categories, shopping time, and recency patterns.
- *Fixed-Length Sequence Preparation:* The project prepared user purchase histories as fixed-length sequences with a maximum sequence length of 50. Shorter sequences were padded, while longer sequences were truncated to preserve recent purchase behavior. This standardization made the data compatible with neural network batch training and helped control computational complexity.
- *LSTM and GRU Attention-Based Models:* The project implemented LSTM + ReLU + Attention and GRU + ReLU + Attention models for next-product prediction. These models learn sequential dependencies from customer purchase histories, while the attention mechanism helps identify the most relevant previous interactions. The final results showed that these recurrent attention models performed strongest among the tested architectures.
- *Hybrid Activation Model Variants:* The project implemented hybrid activation models, including GRU + ReLU-SwiLU and LSTM + ReLU-GELU. These variants were used to test whether smoother activation functions could improve learning compared with standard ReLU. The final results showed that the hybrid activation models were useful experimental comparisons, but they did not outperform the standard LSTM + ReLU and GRU + ReLU models.

- *Transformer Encoder Model Comparison:* The project included a Transformer Encoder + GELU model to compare self-attention-based modeling against recurrent sequence modeling. The Transformer model provided an additional architecture benchmark and helped strengthen the experimental design. In the final comparison, the Transformer model performed below the recurrent attention models, suggesting that the LSTM and GRU architectures were more suitable under the project’s training setup.
- *Classification and Top-K Recommendation Evaluation:* The project evaluated models using both classification and recommendation metrics. Classification metrics included accuracy, weighted precision, weighted recall, and weighted F1-score. Recommendation metrics included Precision@5, Precision@10, Recall@5, Recall@10, Hit Rate@5, and Hit Rate@10. This dual evaluation approach provides both exact prediction performance and realistic recommendation quality.
- *Best Model Identification:* The final comparison identified **LSTM + ReLU + Attention** as the best-performing model. It achieved the highest Recall@10 value of **0.146892**, meaning the true next product appeared in the model’s top 10 recommendations for approximately **14.69%** of test cases. The **GRU + ReLU + Attention** model followed closely with Recall@10 of **0.145853**.
- *Final Consolidated Results Framework:* The project created a final result consolidation framework that loads saved CSV and JSON outputs, removes duplicate rows, standardizes model names, and generates final comparison tables and plots. This makes the results report-ready and allows all models to be compared consistently using Recall@10, Precision@10, training curves, and final summary outputs

To summarize the project setup, the Table presents the key introductory details used throughout this study. These details define the dataset, prediction task, model input size, and practical business relevance of the proposed next-product recommendation system.

Project Detail	Description
Dataset name	Instacart Market Basket Dataset
Problem type	Multi-class classification
Product vocabulary size	25,001 products
Maximum sequence length	50 previous purchase events
Real-world business impact	This project has strong real-world relevance for e-commerce and online grocery platforms because it supports personalized product recommendations, basket completion, cross-selling, customer retention, targeted promotions, inventory planning, and retail customer behavior analysis. By learning from each customer’s historical purchase sequence and temporal shopping patterns, the model can recommend products that

	are more relevant and timely. This can improve shopping convenience, increase customer engagement, support repeat purchases, and help retailers make better business decisions based on predicted product demand and customer behavior.
--	---

Methodology

The methodology of this project follows an end-to-end deep learning pipeline for developing a **time-aware next-product recommendation system** using the Instacart Market Basket dataset. The project begins with raw grocery transaction files stored in Google Drive, performs exploratory data analysis, cleans and integrates multiple relational tables, constructs user-level purchase sequences, engineers product/category/temporal features, prepares fixed-length batches, trains multiple neural network models, and finally evaluates them using classification and Top-K recommendation metrics.

The implemented methodology is divided into five major phases: data understanding, preprocessing and sequence construction, feature engineering and batch generation, model training, and final evaluation. This structure follows the actual notebook workflow: **Phase 1** performs data understanding and EDA, **Phase 2/3** creates clean model-ready sequence batches, **Phase 4** trains neural recommendation models, and the results notebook consolidates outputs into report-ready comparison tables and plots.

Overall Workflow

The overall methodology is designed to transform raw Instacart transaction data into a supervised next-product prediction problem. Each user's purchase history is treated as a chronological sequence. Previous purchase events are used as model input, and the next product becomes the prediction target.

The project uses the required Instacart files: aisles.csv, departments.csv, products.csv, orders.csv, order_products__prior.csv, and order_products__train.csv. The Phase 1 notebook verifies that all required files are available, loads them from Google Drive, and explores their shapes, datatypes, and missing values.

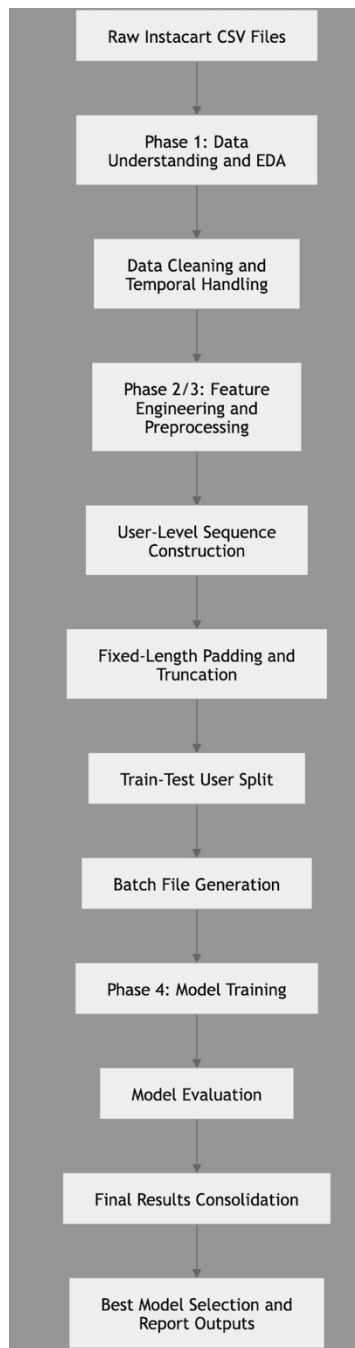


Figure 4.1. Overall methodology workflow for the Instacart next-product recommendation system.

The above figure is an end-to-end workflow for building the time-aware next-product recommendation system from raw data to final model evaluation.

Data Acquisition and Dataset Understanding

The dataset used in this project is the **Instacart Market Basket dataset**, which contains customer order histories and product metadata. The dataset is suitable for sequential recommendation because it includes user order numbers, product purchases, product hierarchy, and time-related order information.

The main files used in the project are summarized below:

File	Role in Methodology
orders.csv	Provides user ID, order number, order day of week, order hour, and days since prior order
order_products__prior.csv	Provides historical products purchased in prior orders
order_products__train.csv	Provides train-order product records
products.csv	Provides product IDs and links products to aisle and department IDs
aisles.csv	Provides aisle-level product categories
departments.csv	Provides department-level product categories

The Phase 1 output shows that the dataset contains **3,421,083 order records**, **32,434,489 prior order-product records**, **1,384,617 train order-product records**, **49,688 products**, **134 aisles**, and **21 departments**. The `orders.csv` table originally contains **206,209 missing values** in `days_since_prior_order`, which correspond to first orders where no previous order exists.

Exploratory Data Analysis

Exploratory Data Analysis is performed to understand user behavior, product popularity, shopping timing, and reorder patterns before model development. This step helps confirm that grocery shopping behavior is sequential and time-dependent. The Phase 1 analysis shows that users have an average of approximately **16.59 orders**, with a median of **10 orders** and a maximum of **100 orders**. This indicates that many users have repeated order histories suitable for sequence modeling. The overall reorder rate is **0.5897**, showing that a large portion of grocery purchases are repeated, which supports the use of next-product recommendation. The EDA also visualizes day-of-week patterns, hour-of-day patterns, top products such as bananas and organic produce, department-level sales, and days between orders. The figure below is an overview of exploratory data analysis steps used to understand user behavior, product trends, and temporal patterns.

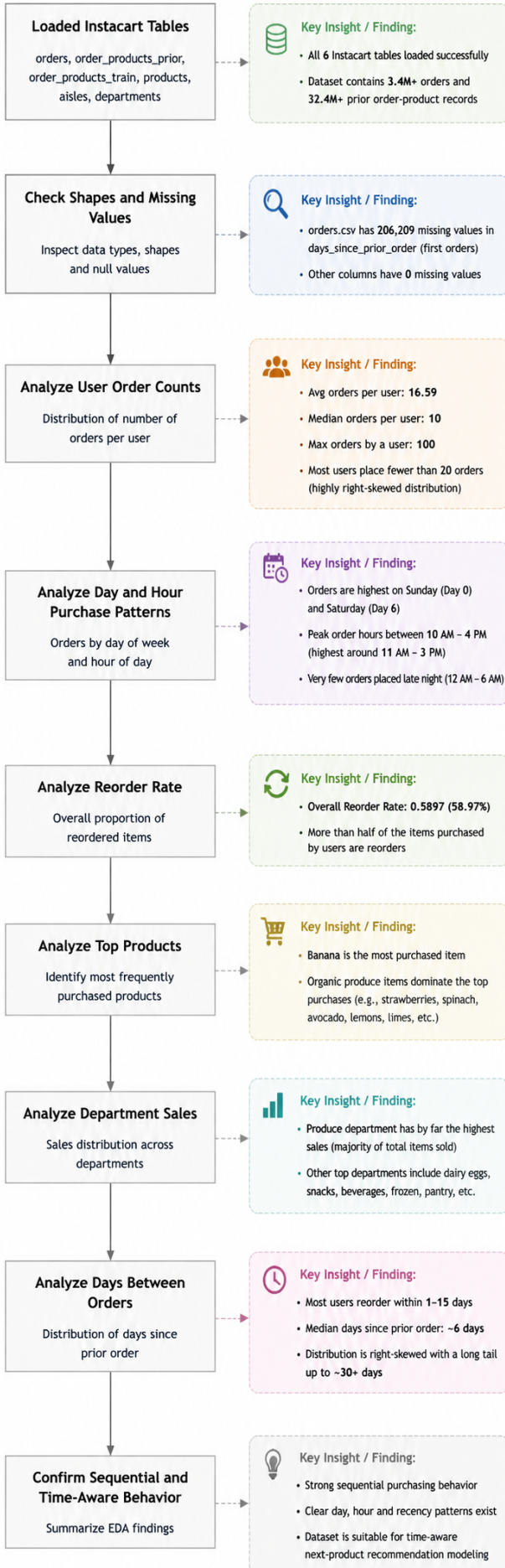


Figure 4.2. Exploratory data analysis workflow and key insights.

Data Cleaning and Temporal Handling

The most important cleaning step is handling missing values in `days_since_prior_order`. These missing values occur because the first order for each user has no prior order. Instead of dropping these rows, the project creates a first-order flag and fills missing values with zero.

The notebook first identifies missing values, then creates: `is_first_order = 1` if `order_number == 1`, otherwise 0

The `days_since_prior_order` column is then filled with 0 for first orders. The final validation confirms that there are **0 missing values** after the fix and **206,209 first-order records**, matching the number of unique users.

This step is important because it preserves the meaning of first orders while allowing the recency feature to be used in neural network training.

The figure below explains the process of handling missing temporal values and identifying first-order transactions for accurate recency modeling.

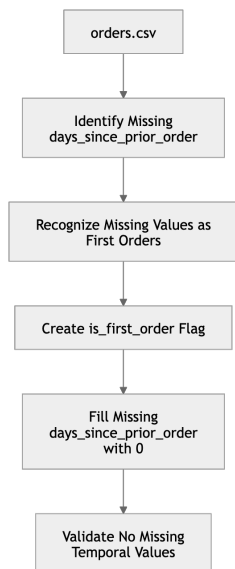
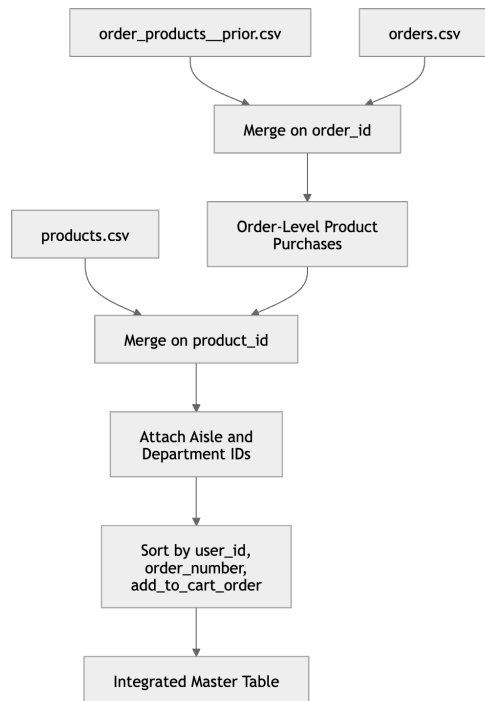


Figure 4.3. Temporal cleaning workflow.

Data Integration

After cleaning, the project integrates multiple Instacart tables into a unified purchase-level dataset. The prior order-product table is merged with the orders table using `order_id`, which attaches user ID, order number, day of week, hour of day, and days since prior order to each product purchase. The product table is then merged using `product_id`, which attaches aisle and department information.

The figure below explains the process of merging multiple Instacart tables to create a unified purchase-level dataset with product and category information.



4.4 Data integration workflow.

The integrated master table contains the following key fields:

Field	Purpose
<code>user_id</code>	Identifies the customer
<code>order_id</code>	Identifies the order
<code>order_number</code>	Preserves chronological order
<code>product_id</code>	Identifies the purchased product
<code>add_to_cart_order</code>	Preserves item order within basket
<code>order_dow</code>	Captures day-of-week shopping pattern

order_hour_of_day	Captures time-of-day shopping pattern
days_since_prior_order	Captures recency and time gap
aisle_id	Captures product group/category
department_id	Captures broader product category

The Phase 3 preprocessing notebook builds this master table by merging prior purchases, order metadata, and product metadata, then sorts the data by `user_id`, `order_number`, and `add_to_cart_order` to preserve chronological sequence order.

Feature Engineering and Encoding

The project engineers product, category, and temporal features for each purchase event. Categorical variables are encoded into integer IDs so they can be passed into embedding layers during model training.

The engineered features include:

Feature	Encoded Name	Description
Product ID	product_enc	Encoded product purchased by the user
Aisle ID	aisle_enc	Encoded product aisle category
Department ID	dept_enc	Encoded product department category
Day of week	dow_enc	Encoded day of week when order was placed
Hour of day	hour_enc	Encoded hour when order was placed
Days since prior order	days_enc	Recency/time-gap feature
First order flag	first_order_enc	Indicates first order of a user

The Phase 3 notebook builds `aisle2id` and `dept2id` mappings, maps product IDs using `product2id`, and validates that the encoded columns contain no missing or negative values. Each purchase event can be represented as:

$$x_t = [product_t, aisle_t, department_t, dow_t, hour_t, days_t]$$

where x_t is the purchase event at time step t .

User-Level Sequence Construction

The core methodology step is converting purchases into user-level event sequences. For each user, the project creates aligned lists of encoded products, aisles, departments, days of week, hours of day, and days since prior order.

The Phase 3 notebook constructs a `user_sequences` dictionary where each user has the following aligned sequence keys:

p = product sequence
a = aisle sequence
d = department sequence
dow = day-of-week sequence
hr = hour-of-day sequence
days = days-since-prior-order sequence

The code validates that all feature sequences for each user are aligned and have the same length.

The supervised learning formulation is:

$$X_i = [x_1, x_2, \dots, x_t]$$
$$y_i = x_{t+1}$$

where previous purchase events form the input sequence and the next product is the target label.

The figure below demonstrates the transformation of purchase records into chronological user-level sequences for supervised next-product prediction.

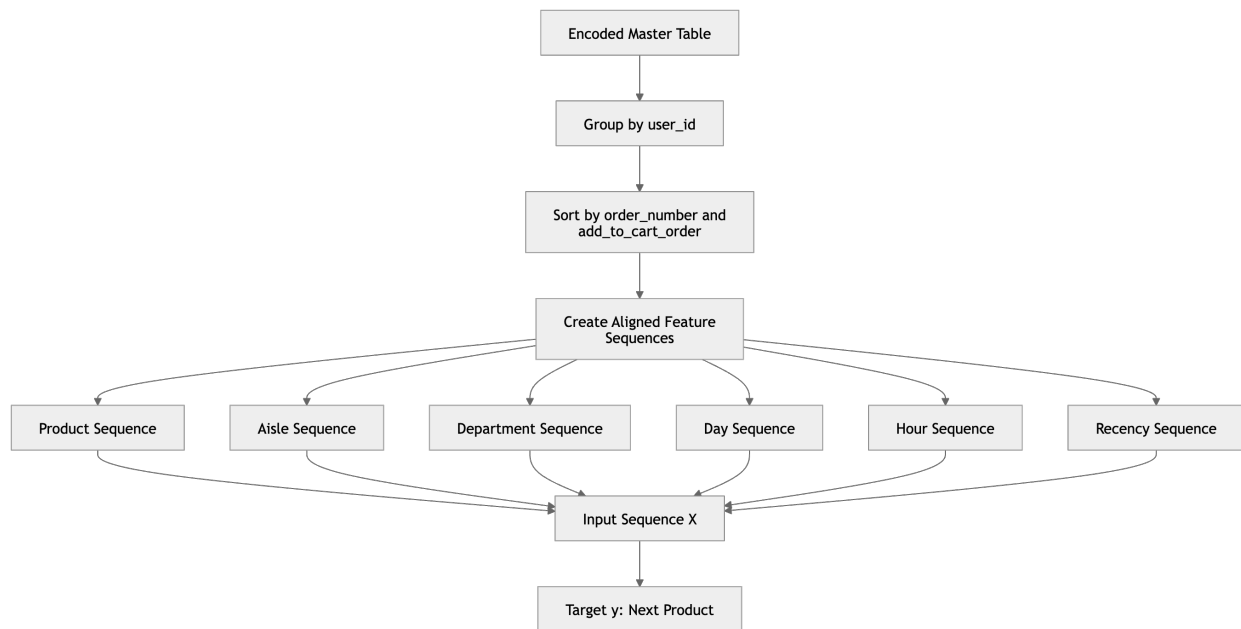


Figure 4.5. User-level sequence construction workflow.

Fixed-Length Padding, Truncation, and Batch Generation

Because users have different sequence lengths, all input sequences are converted to a fixed length of **50**. The project uses a NumPy-based padding function that applies **left truncation** and **left padding**. This means the model keeps the most recent 50 events when a sequence is too long and pads shorter sequences with zeros. The Phase 3 preprocessing creates the following final tensors:

The figure below explains the Padding and truncation strategy used to convert variable-length user sequences into fixed-length model inputs.

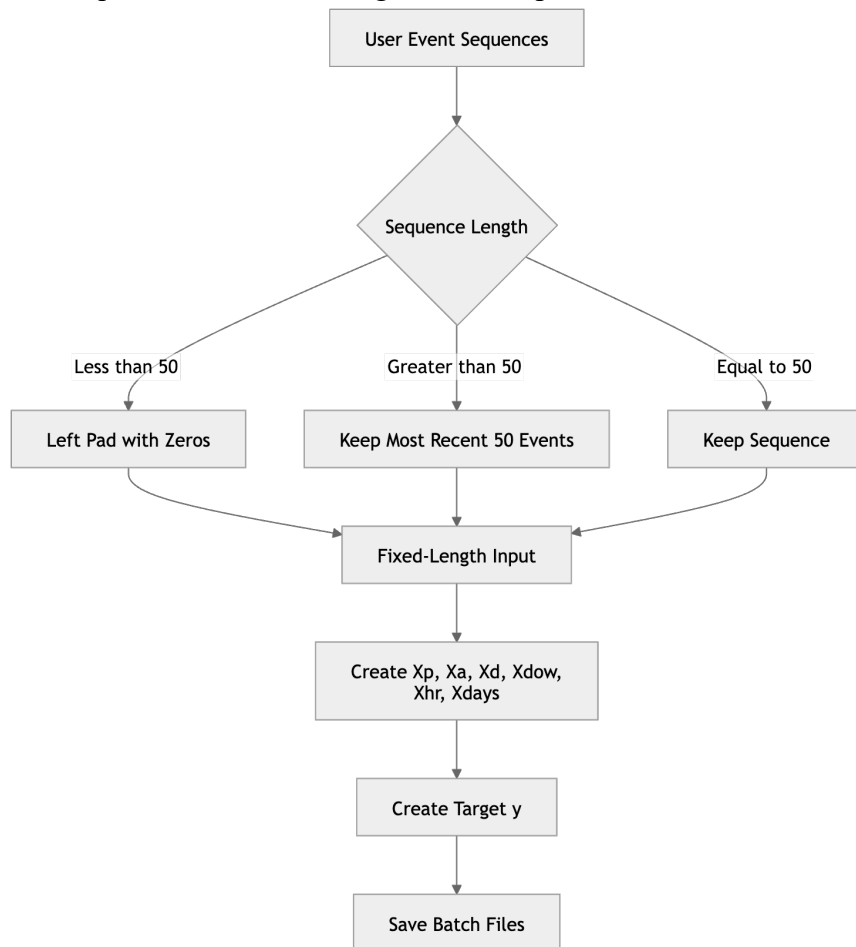


Figure 4.6. Fixed-length sequence and batch generation workflow.

Tensor	Meaning	Shape
Xp	Product sequence input	(batch_size, 50)
Xa	Aisle sequence input	(batch_size, 50)
Xd	Department sequence input	(batch_size, 50)
Xdow	Day-of-week sequence input	(batch_size, 50)
Xhr	Hour-of-day sequence input	(batch_size, 50)
Xdays	Days-since-prior-order sequence input	(batch_size, 50)
y	Target next product	(batch_size,)

The final Phase 3 summary reports **MAX_LEN = 50, CHUNK_SIZE = 100, 164,331 train users, 41,083 test users, 1,644 train batches, 411 test batches, 25,001 product vocabulary size, 134 aisle vocabulary size, and 21 department vocabulary size.**

Train-Test Split

The project uses a user-level train-test split to prevent leakage. This means the same user does not appear in both training and testing sets. The Phase 3 notebook validates that there is no overlap between train users and test users and confirms that the <UNK> token exists in the product vocabulary and maps to 0. This split makes the evaluation more realistic because the trained model is tested on users not used during training.

Component	Value
Train users	164,331
Test users	41,083
Total users	205,414
Product vocabulary size	25,001
Maximum sequence length	50
Train batches	1,644
Test batches	411

Model Development and Training Procedure

After preprocessing, the processed batch files are used for model training. The project trains and compares several recommendation models under the same time-aware sequential framework.

The figure below shows the Neural network training pipeline including embedding, sequence modeling, attention mechanism, and optimization.

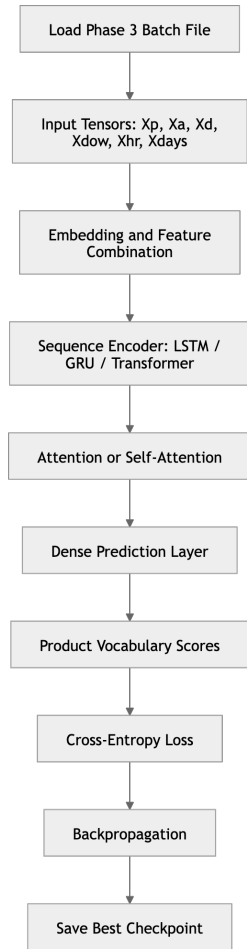


Figure 4.7. Model training workflow.

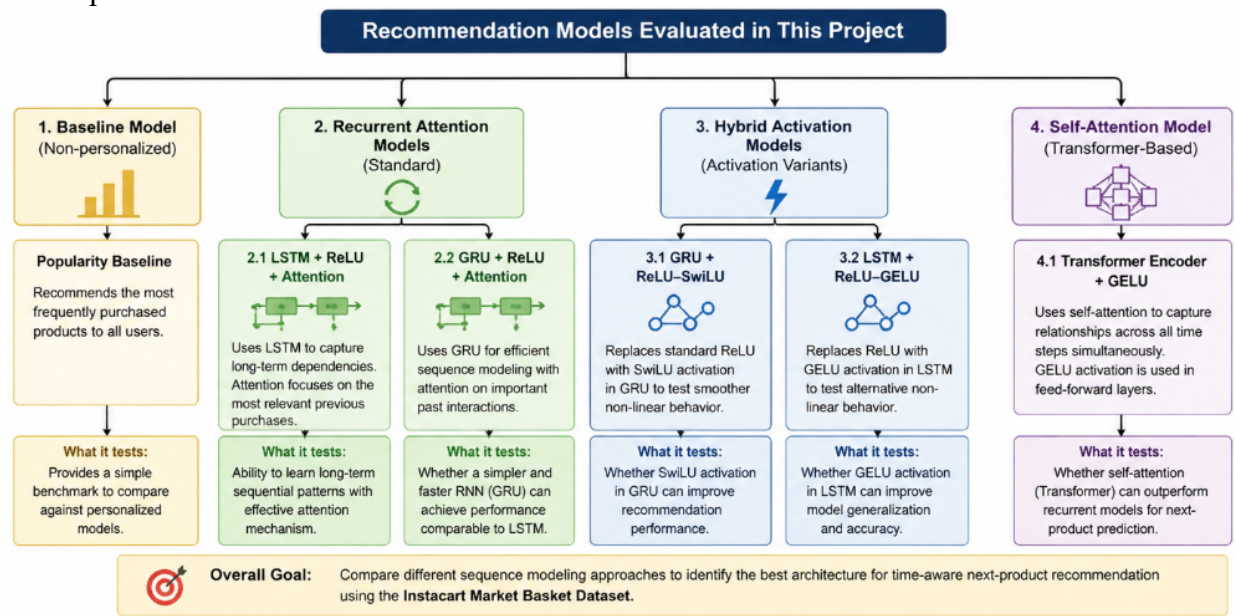
The neural models use product, aisle, department, day, hour, and recency features as input. Categorical features are passed through embedding layers, temporal features are added as numerical/contextual inputs, and the resulting sequence is processed using LSTM, GRU, or Transformer Encoder layers. The recurrent models use attention to summarize the most relevant previous purchases.

Training is supervised: the input is the user's previous sequence and the label is the next product. The model is optimized using cross-entropy loss:

$$L = - \sum_{i=1}^C y_i \log (\hat{y}_i)$$

where C is the number of product classes, y_i is the true target label, and $\hat{y}_i$ is the predicted probability.

The implemented models are:



Evaluation Metrics

The models are evaluated using both classification metrics and recommendation-specific Top-K metrics.

Classification metrics include:

Metric	Purpose
Accuracy	Measures exact next-product prediction correctness
Weighted Precision	Measures precision while accounting for product imbalance
Weighted Recall	Measures recall while accounting for product imbalance
Weighted F1-score	Balances weighted precision and weighted recall

Top K Metrics include:

Metric	Purpose
Recall@5	Checks whether the true next product appears in the top 5 recommendations
Recall@10	Checks whether the true next product appears in the top 10 recommendations
Precision@5	Measures top-5 recommendation precision
Precision@10	Measures top-10 recommendation precision
Hit Rate@5	Measures the top-5 hit percentage
Hit Rate@10	Measures the top-10 hit percentage

For this project, the task is single-label next-product prediction. Therefore, if the true next product appears in the Top-K list, it is counted as a hit, making Recall@K equivalent to Hit Rate@K.

$$Recall@K = HitRate@K$$

$$Precision@K = \frac{HitRate@K}{K}$$

Final Results Consolidation

The final results notebook searches saved model result files, loads CSV and JSON outputs, removes duplicate model rows, standardizes model names, and creates a final comparison table. It also generates Recall@10 plots, Precision@10 plots, and the training curve for the best model. The final comparison table includes:

Model	Type	Epochs Run	Best Epoch	Best Val F1	Recall@5	Recall@10	Precision@5
LSTM + ReLU	Recurrent + Attention	50	49	0.013982	0.101161	0.146892	0.020232
GRU + ReLU	Recurrent + Attention	20	16	0.013281	0.100307	0.145853	0.020061
GRU + ReLU-SwiLU	Recurrent + Hybrid Activation	5	5	0.011495	0.097549	0.141544	0.019510

LSTM + ReLU-GELU	Recurrent + Hybrid Activation	5	5	0.011758	0.096233	0.139560	0.019247
Transformer Encoder + GELU	Self-Attention	5	5	0.010160	0.094174	0.135580	0.018835

The final summary identifies **LSTM + ReLU** as the best-performing model, with **Recall@10 = 0.146892**, meaning that the true next product appeared in the model's top 10 recommendations for approximately **14.69%** of test cases. The GRU + ReLU model performed very closely, while hybrid activation variants and the Transformer Encoder did not outperform the standard ReLU-based recurrent models. The below figure shows the Consolidation of model outputs into a unified comparison framework for selecting the best-performing model.

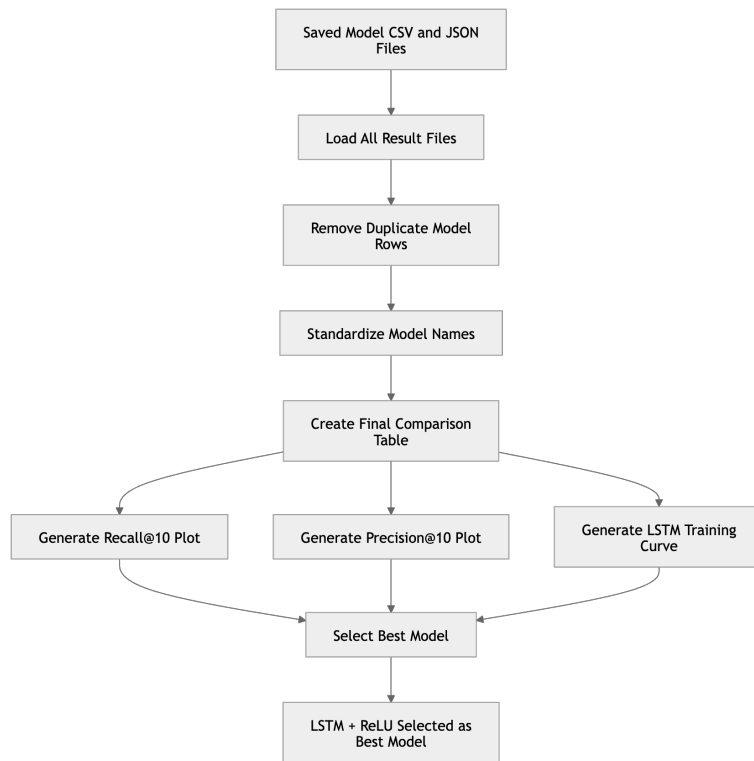


Figure 4.8. Final result consolidation and best model selection workflow.

Methodology Summary

In summary, the methodology transforms raw Instacart transaction records into a complete time-aware sequential recommendation pipeline. Phase 1 performs data understanding and EDA, including dataset exploration, reorder analysis, product popularity analysis, department analysis, and time-based order analysis. Phase 2/3 prepares model-ready data by handling missing time values, merging product and order information, encoding product/category/time features, constructing user-level sequences, applying fixed-length padding and truncation, and saving train/test batches. Phase 4 trains multiple recommendation models, and the final output notebook consolidates all results into report-ready comparison tables and plots.

The methodology is appropriate for this project because next-product recommendation in grocery shopping depends strongly on purchase order, product repetition, category relationships, and temporal patterns. By combining sequence modeling, product hierarchy, time-aware features, and Top-K evaluation, the project creates a practical and scalable framework for personalized next-product recommendation.

Model Description

The model description section explains how the proposed recommendation models are structured, how they learn from user purchase sequences, and why each design choice was selected. Since the task is to predict the next product from a customer's historical purchases, the project uses a time-aware sequential architecture. The models receive product, category, and temporal features as input, encode them using embeddings, process the purchase history using sequence models, and generate Top-K next-product recommendations.

Overall Model Design

The recommendation task is formulated as a **multi-class classification problem**, where each product in the vocabulary is treated as a possible output class. For each user, the model receives a fixed-length sequence of previous purchase events and predicts the next product.

Each purchase event is represented as:

$$x_t = [p_t, a_t, d_t, dow_t, hr_t, \Delta_t]$$

where p_t is the product ID, a_t is the aisle ID, d_t is the department ID, dow_t is the day of week, hr_t is the hour of day, and Δ_t is the number of days since the previous order.

The model input is:

$$X = [x_1, x_2, \dots, x_T]$$

where:

$$T = 50$$

The target output is:

$$y = p_{T+1}$$

This means the model learns a mapping from a user's previous purchase sequence to the next product:

$$\hat{y} = f_{\theta}(X)$$

where f_{θ} is the neural network and $\hat{y}$ is the predicted probability distribution over the product vocabulary.

The overall architecture follows this flow:



Figure 5.1. Overall architecture of the time-aware next-product recommendation model.

The overall architecture of the time-aware recommendation model showing how user purchase sequences are transformed into Top-K next-product recommendations is the above figure. This structure is used because grocery shopping is sequential and time-dependent. The model needs to know not only which products were purchased, but also the order of purchase, the product categories, and the time gap between purchases.

Input Representation and Feature Embeddings

The model uses a **multi-input representation** instead of a product-only sequence. This is important because grocery recommendation depends on several types of information. A product ID captures the exact item, aisle and department capture product hierarchy, and day, hour, and recency capture shopping time behavior.

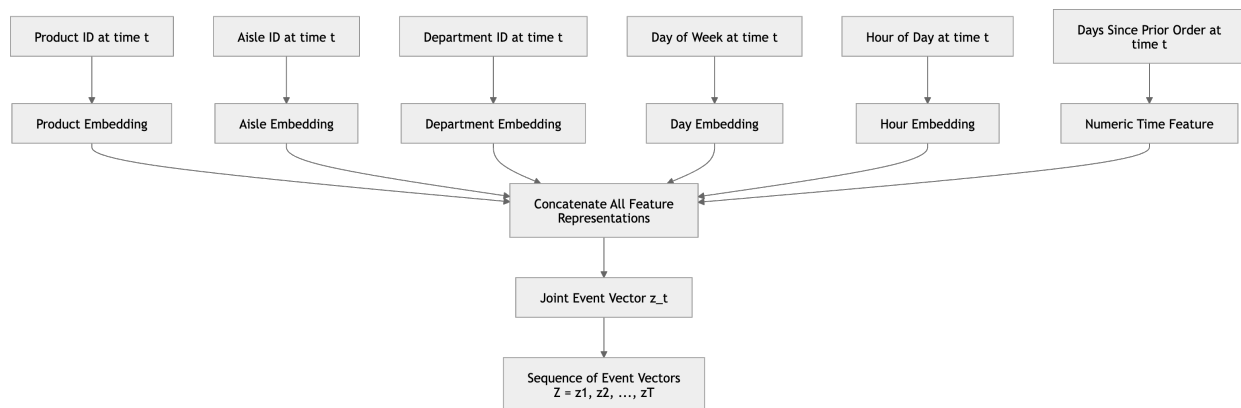


Figure 5.2. Multi-input feature representation and embedding design for each purchase event.

Figure 5.2. Multi-input embedding design showing how product, category, and temporal features are combined into a unified purchase-event representation.

The model receives six aligned input tensors:

Input	Meaning	Purpose
X_p	Product sequence	Captures exact product history
X_a	Aisle sequence	Captures product group behavior
X_d	Department sequence	Captures broader category behavior
X_{dow}	Day-of-week sequence	Captures weekly shopping pattern

X_{hr}	Hour-of-day sequence	Captures time-of-day behavior
X_{Delta}	Days-since-prior-order sequence	Captures recency and purchase gap

Each tensor has the shape: $(batch_size, 50)$

The categorical inputs are passed through embedding layers:

$$\begin{aligned}
E_p &= Emb_p(X_p) \\
E_a &= Emb_a(X_a) \\
E_d &= Emb_d(X_d) \\
E_{dow} &= Emb_{dow}(X_{dow}) \\
E_{hr} &= Emb_{hr}(X_{hr})
\end{aligned}$$

The recency feature is treated as a numerical feature and combined with the embedded categorical features. At each time step, the feature vector is constructed as:

$$z_t = [E_p(t) \parallel E_a(t) \parallel E_d(t) \parallel E_{dow}(t) \parallel E_{hr}(t) \parallel X_{\Delta}(t)]$$

where $\parallel$ means concatenation.

Embeddings are used because the product vocabulary is large. A one-hot encoded product vector would be sparse and inefficient, especially with 25,001 products. Embeddings reduce dimensionality and allow the model to learn product similarity patterns during training.

Sequence Modeling Architectures

After feature concatenation, the sequence representation is passed into a sequence encoder. The project compares three types of sequence models: LSTM, GRU, and Transformer Encoder.

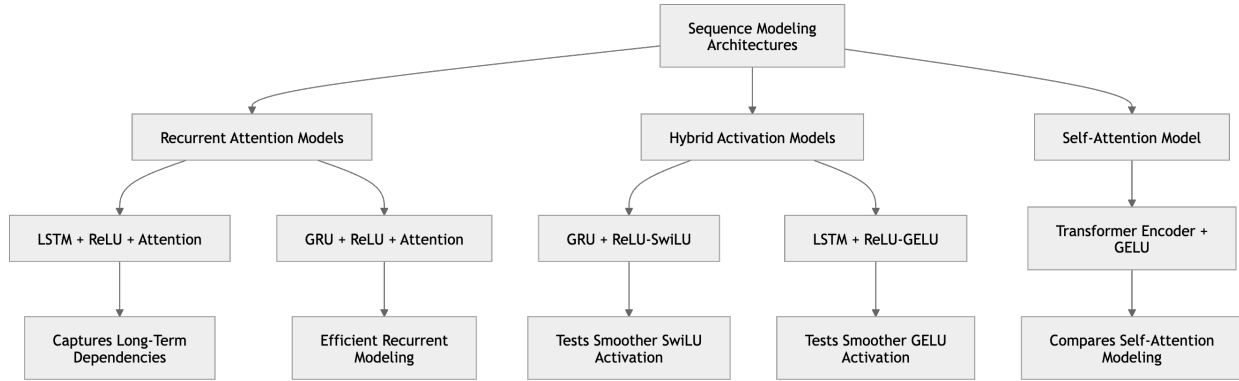


Figure 5.3. Comparison of sequence modeling architectures evaluated in the project.

The figure shows the comparison of the sequence modeling approaches evaluated, including recurrent attention models, hybrid activation models, and the Transformer Encoder.

LSTM-Based Architecture

The LSTM model is used to capture long-term dependencies in purchase histories. It is useful when earlier purchases may still influence future purchases. The LSTM maintains a memory cell and uses input, forget, and output gates to control information flow.

The LSTM processes the embedded sequence and produces hidden states:

$$H = [h_1, h_2, \dots, h_T]$$

The LSTM was chosen because grocery behavior often contains both short-term and long-term patterns. For example, some items may be repurchased weekly, while others may follow longer cycles. The memory structure of LSTM helps preserve useful information across the sequence.

GRU-Based Architecture

The GRU model is used as a lighter recurrent alternative to LSTM. It uses update and reset gates but has fewer parameters than LSTM. This makes it computationally efficient while still being capable of learning sequence dependencies.

GRU was included because the Instacart dataset is large, and a simpler recurrent model can reduce training complexity. In the final results, the GRU model performed very close to the LSTM model, showing that it is an effective alternative for this task.

Transformer Encoder Architecture

The Transformer Encoder model is included as a self-attention-based comparison. Unlike LSTM and GRU, the Transformer does not process the sequence step by step. Instead, it uses self-attention to compare all sequence positions directly.

The self-attention mechanism is:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The Transformer was included to compare recurrent sequence modeling with self-attention-based modeling. Although Transformer models can capture global sequence relationships, the lightweight Transformer variant performed slightly lower than the recurrent models in this project, likely because it required more training, tuning, or computational resources.

Attention and Prediction Layer

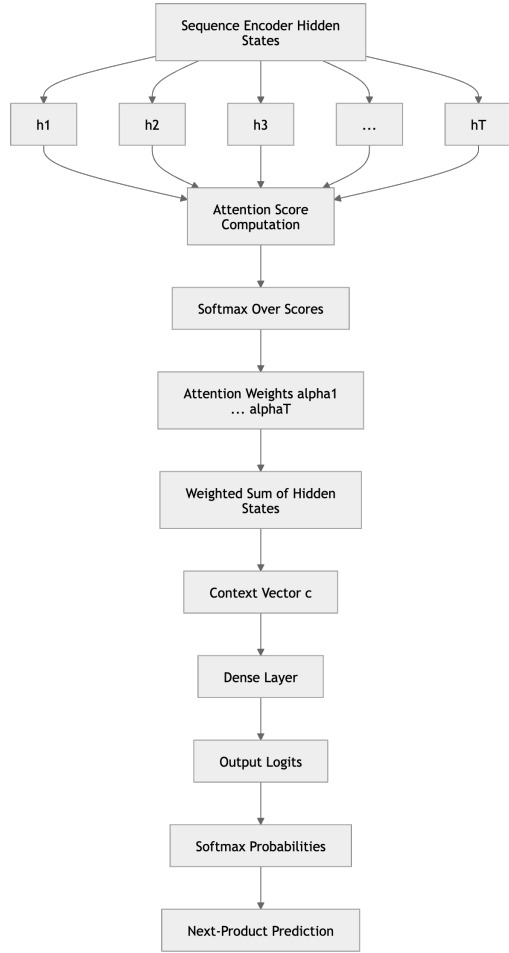


Figure 5.4. Attention Mechanism

The above figure shows the attention mechanism used to assign importance weights to previous purchases and create a context vector for next-product prediction. For the LSTM and GRU models, an attention mechanism is applied after the recurrent encoder. Attention is used because not every previous purchase contributes equally to the next-product prediction. Some recent or related purchases may be more important than older or unrelated purchases. Given hidden states:

$$H = [h_1, h_2, \dots, h_T]$$

attention computes a score for each time step:

$$e_t = v^T \tanh(W_a h_t + b_a)$$

The scores are converted into attention weights:

$$\alpha_t = \frac{\exp(e_t)}{\sum_{j=1}^T \exp(e_j)}$$

The final context vector is calculated as:

$$c = \sum_{t=1}^T \alpha_t h_t$$

This context vector summarizes the most important parts of the user's purchase history. It is passed into dense layers and then mapped to the product vocabulary:

$$o = W_o c + b_o$$

where o represents product logits. The logits are converted into probabilities using softmax:

$$\hat{y}_i = \frac{\exp(o_i)}{\sum_{j=1}^C \exp(o_j)}$$

where:

$$C = 25,001$$

The model then ranks products by predicted probability and selects Top-5 and Top-10 recommendations.

Training Algorithm and Loss Function

The model is trained using supervised learning. Each sample contains a fixed-length user sequence and one true next-product label. The model predicts a probability distribution across all products, and the loss function penalizes incorrect predictions. The training loss is cross-entropy:

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

Since each sample has one true next product, this simplifies to:

$$L = -\log(\hat{y}_{true})$$

Cross-entropy is appropriate because the task is multi-class classification over a large product vocabulary.

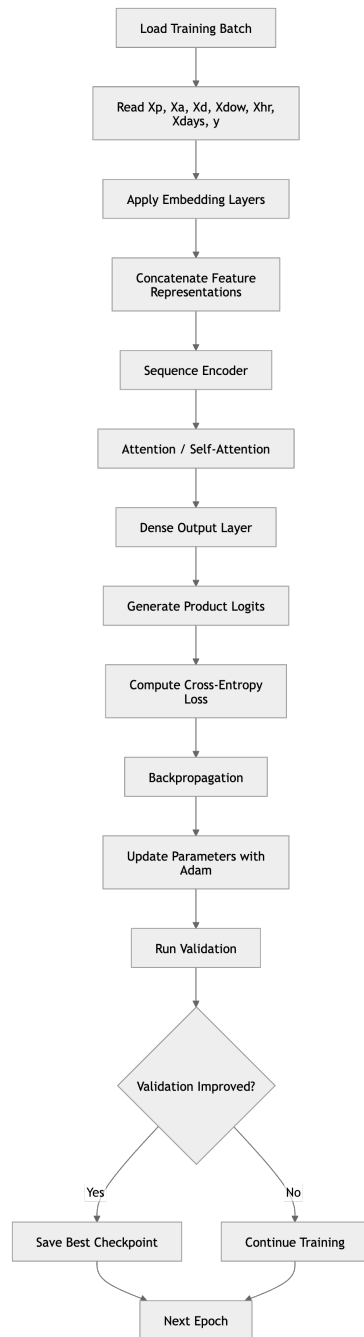


Figure 5.5. Training Workflow

The training algorithm follows these steps:

1. Load a processed training batch.
2. Read product, aisle, department, day, hour, and recency sequences.
3. Pass categorical inputs through embedding layers.
4. Concatenate embeddings with recency features.
5. Process the sequence using LSTM, GRU, or Transformer Encoder.
6. Apply attention or self-attention.
7. Generate product logits over the vocabulary.
8. Compute cross-entropy loss against the true next product.
9. Backpropagate gradients.
10. Update model weights using the optimizer.
11. Validate model performance.
12. Save the best checkpoint.

During evaluation, the trained model generates product scores for each test sequence. Products are ranked, and the model is evaluated using Precision@5, Precision@10, Recall@5, Recall@10, Hit Rate@5, and Hit Rate@10.

Model Variants Compared

The project compares multiple model variants to evaluate which architecture is most suitable for the next-product recommendation task.

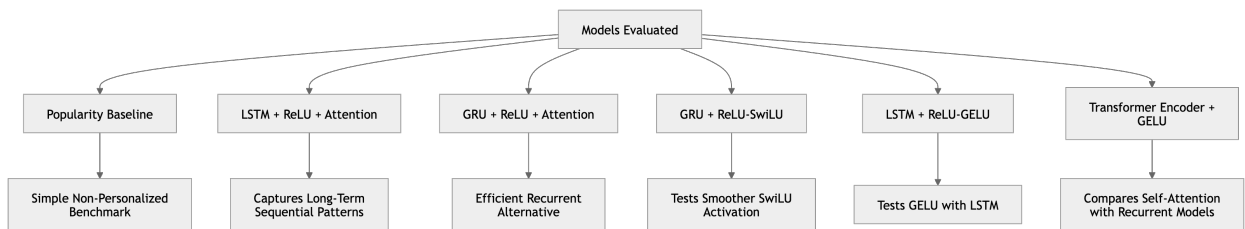


Figure 5.7. Model Variants and Design Rationale

Model	Architecture Type	Design Purpose
Popularity Baseline	Non-personalized baseline	Recommends most frequent products and provides a simple benchmark

LSTM + ReLU + Attention	Recurrent + Attention	Captures longer-term purchase dependencies
GRU + ReLU + Attention	Recurrent + Attention	Provides efficient sequence modeling with fewer parameters
GRU + ReLU-SwiLU	Recurrent + Hybrid Activation	Tests smoother SwiLU activation with GRU
LSTM + ReLU-GELU	Recurrent + Hybrid Activation	Tests GELU activation with LSTM
Transformer Encoder + GELU	Self-Attention	Compares Transformer-style sequence modeling against recurrent models

The hybrid activation models were included to test whether smoother nonlinear functions improve learning. ReLU is efficient and stable, while SwiLU and GELU are smoother activations that may improve gradient flow in some neural networks. However, the final results showed that the standard ReLU-based recurrent attention models performed better than the hybrid variants.

The popularity baseline was included to ensure that the neural models are compared against a simple and practical benchmark. Since grocery datasets often contain frequently repeated products, popularity-based recommendation can be competitive, so it is important to test whether personalized sequential models add value.

Design Rationale

The architecture was designed based on the characteristics of grocery shopping behavior.

Design Choice	Reason
Fixed sequence length of 50	Standardizes input size and focuses on recent behavior
Product embeddings	Learn dense product relationships instead of sparse one-hot vectors
Aisle and department embeddings	Capture category-level shopping preferences
Day and hour inputs	Learn weekly and time-of-day shopping patterns
Recency feature	Captures time gaps and repeat-purchase cycles
LSTM	Learns longer-term sequential dependencies
GRU	Provides efficient recurrent sequence modeling
Attention	Focuses on the most relevant previous purchases
Transformer Encoder	Tests self-attention as an alternative to recurrence

Cross-entropy loss	Suitable for multi-class next-product prediction
Top-K evaluation	Matches real recommendation systems that show ranked suggestions

The final model comparison showed that the recurrent attention models were most suitable for this dataset. LSTM + ReLU + Attention achieved the strongest Top-K performance, while GRU + ReLU + Attention performed very closely. This suggests that recurrent models with attention were able to capture the sequential and temporal structure of Instacart purchase behavior more effectively than the tested hybrid activation and Transformer variants.

Experiments and Results

This section presents the experimental setup, dataset description, training and testing process, performance metrics, and comparison of the evaluated models. The goal of the experiments is to determine which model architecture performs best for the time-aware next-product recommendation task using the Instacart Market Basket dataset.

Database / Dataset Description

The dataset used in this project is the **Instacart Market Basket Dataset**, which contains customer grocery order histories, product information, and product-category metadata. This dataset is suitable for next-product recommendation because it captures repeated customer purchases over time and includes temporal order information such as order number, day of week, hour of day, and days since the prior order. The main files used in the project are:

Dataset File	Description
orders.csv	Contains order-level information such as <code>order_id</code> , <code>user_id</code> , <code>order_number</code> , <code>order_dow</code> , <code>order_hour_of_day</code> , and <code>days_since_prior_order</code> .
order_products__prior.csv	Contains products purchased in users' prior orders and is used to build historical purchase sequences.
order_products__train.csv	Contains products from users' training orders.
products.csv	Contains product metadata including <code>product_id</code> , <code>product_name</code> , <code>aisle_id</code> , and <code>department_id</code> .
aisles.csv	Contains aisle-level product category information.
departments.csv	Contains department-level product category information.

The Phase 1 notebook loaded all required Instacart files from Google Drive and verified their availability. The dataset includes **3,421,083 orders**, **32,434,489 prior order-product records**, **1,384,617 train order-product records**, **49,688 products**, **134 aisles**, and **21 departments**. The original `orders.csv` file contained **206,209 missing values** in `days_since_prior_order`, which correspond to first orders where no previous order exists. These missing values were handled by creating an `is_first_order` flag and filling missing values with 0.

Component	Value
Total orders	3,421,083
Prior order-product records	32,434,489
Train order-product records	1,384,617
Products	49,688
Aisles	134
Departments	21
Users	206,209
Overall reorder rate	0.5897

The exploratory data analysis showed that users had an average of approximately **16.59 orders**, a median of **10 orders**, and a maximum of **100 orders**. This confirms that many users have repeated purchase histories, making the dataset suitable for sequential modeling. The overall reorder rate of **0.5897** also indicates that grocery purchases are highly repetitive, which supports the use of next-product recommendation models.

Exploratory Data Analysis Outputs

Exploratory Data Analysis was performed to understand customer shopping behavior and confirm that the dataset is suitable for sequential recommendation. The analysis included order frequency per user, purchases by day of week, purchases by hour of day, top products, department-level purchases, reorder rate, and days between orders.

The EDA showed that customers have repeated purchase histories, with an average of approximately **[Insert average orders per user: 16.59]** orders per user and a median of **[Insert median orders per user: 10]** orders. The reorder rate of **[Insert reorder rate: 0.5897]** indicates that many grocery purchases are repeated, which supports the use of next-product recommendation

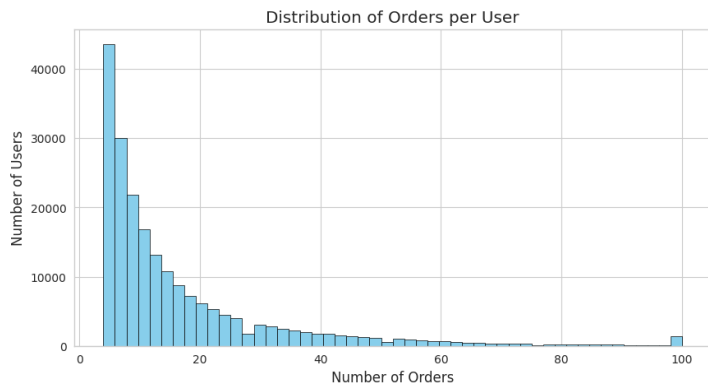


Figure 6.1. Distribution of Orders per User

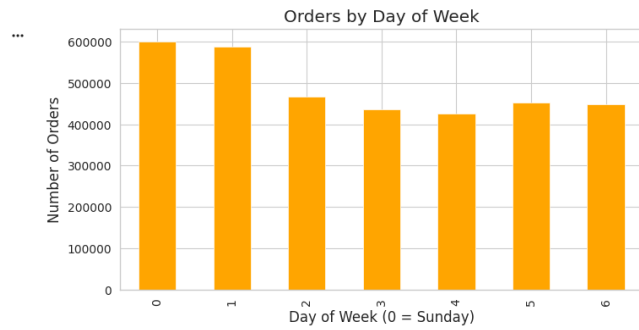


Figure 6.2. Orders by Day of Week

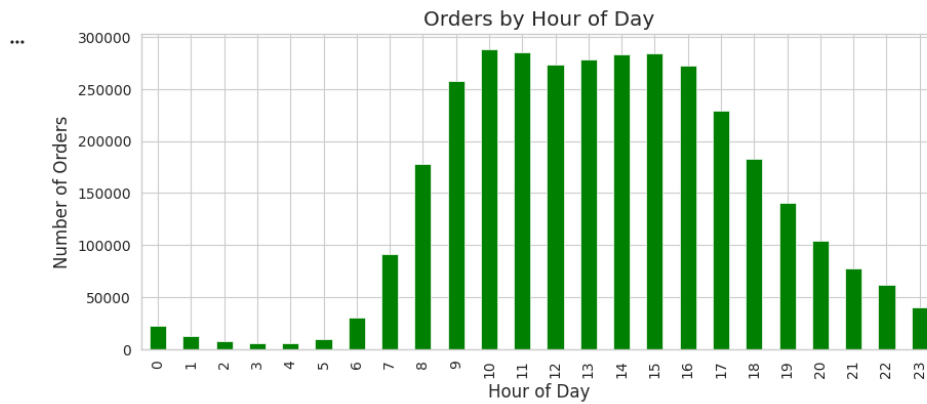


Figure 6.3. Orders by Hour of Day

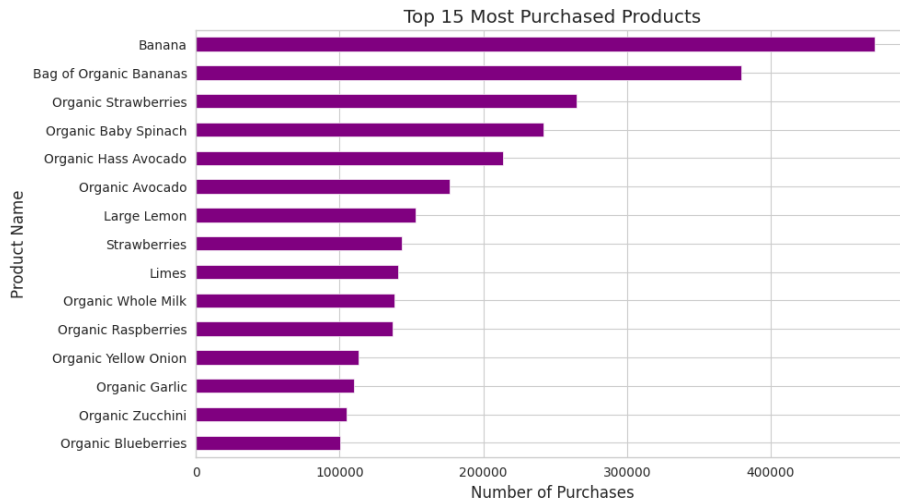


Figure 6.4. Top 15 Most Purchased Product

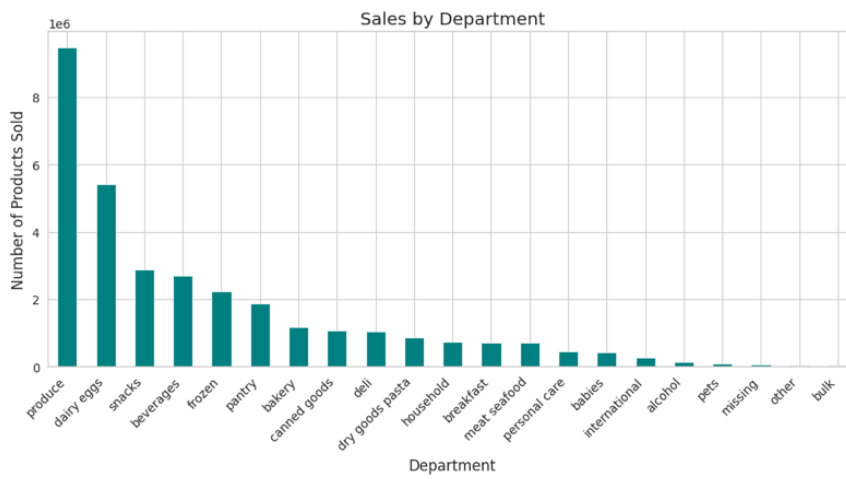


Figure 6.5. Sales by Department

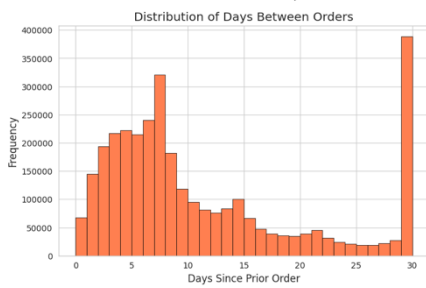


Figure 6.6. Distribution of Days Between Orders

EDA Findings

The exploratory data analysis shows that the Instacart dataset contains strong patterns of repeated and time-dependent grocery shopping behavior. These patterns support the decision to model the problem as a **time-aware sequential recommendation task** rather than a simple product classification problem.

The **distribution of orders per user** is highly right-skewed. Most users place a relatively small number of orders, while a smaller group of users have very long order histories, reaching up to 100 orders. This indicates that customer activity levels vary significantly across users. From a modeling perspective, this justifies using sequence padding and truncation, because users do not all have the same purchase history length. It also supports the use of a fixed maximum sequence length so that shorter histories can be padded and longer histories can be standardized for neural network training.

The **orders by day of week** plot shows that order activity is not uniform across the week. The highest order volumes occur around the beginning of the week, especially on Day 0 and Day 1, while the middle of the week has comparatively lower order activity. This suggests that customers follow weekly shopping routines. Including the day-of-week feature in the model is therefore useful because the timing of an order may influence what product a customer is likely to purchase next.

The **orders by hour of day** plot shows a clear daily shopping pattern. Order volume is very low during late-night and early-morning hours, then increases sharply after morning hours. The highest activity occurs during late morning to afternoon hours, especially between approximately 10 AM and 4 PM. This indicates that hour of day is an important temporal signal. Including the order hour feature allows the model to learn time-of-day shopping behavior and improve next-product prediction.

The **top 15 most purchased products** plot shows that a small number of products dominate purchase frequency. Bananas and organic bananas are the two most frequently purchased products, followed by organic fruits, vegetables, milk, and other common grocery staples. This confirms that the dataset has strong product popularity imbalance. Because some products appear much more frequently than others, exact next-product prediction is difficult, and weighted classification metrics are more appropriate than simple accuracy alone. This also explains why a popularity baseline is important for comparison.

The **sales by department** plot shows that the produce department has the highest number of products sold by a large margin. Other major departments include dairy/eggs, snacks, beverages,

frozen, and pantry. This suggests that customer purchase behavior is strongly influenced by product category. Therefore, using aisle and department features in addition to product IDs is useful because the model can learn both item-level and category-level patterns.

The **distribution of days between orders** shows that many customers reorder within a short time window, especially within the first few days to about one week. There is also a noticeable spike around 30 days, suggesting monthly or scheduled shopping behavior for some users. This confirms that recency is an important factor in grocery shopping. The `days_since_prior_order` feature helps the model learn repeat-purchase cycles and time gaps between orders.

Overall, the EDA confirms three important characteristics of the dataset: customer purchases are **sequential**, **repetitive**, and **time-dependent**. Users show different order frequencies, products show strong popularity imbalance, departments reveal category-level structure, and day/hour/recency patterns show clear temporal behavior. These findings justify the final modeling approach, which uses product, aisle, department, day-of-week, hour-of-day, and days-since-prior-order features in a time-aware sequential neural recommendation system.

Experimental Data Preparation

Before training the models, the raw transaction data was transformed into model-ready user sequences. The preprocessing pipeline merged prior order-product records with order metadata and product metadata. This created an integrated purchase-level table containing product ID, user ID, order number, day of week, hour of day, days since prior order, aisle ID, and department ID.

The project then created encoded features for:

Feature	Encoded Field	Purpose
Product ID	product_enc	Represents the purchased product
Aisle ID	aisle_enc	Represents product group/category
Department ID	dept_enc	Represents broader product category
Day of week	dow_enc	Represents weekly shopping pattern
Hour of day	hour_enc	Represents time-of-day shopping behavior
Days since prior order	days_enc	Represents recency/time gap
First order flag	first_order_enc	Identifies first-order cases

The Phase 3 preprocessing notebook built a master table, encoded product/category/time features, constructed user-level event sequences, and generated model-ready batch files. Each

user sequence contained aligned product, aisle, department, day, hour, and recency sequences. The batch tensors were saved as X_p , X_a , X_d , X_{dow} , X_{hr} , X_{days} , and target y .

Component	Value
Maximum sequence length	50
Product vocabulary size	25,001
Train users	164,331
Test users	41,083
Total users after split	205,414
Train batches	1,644
Test batches	411
Batch chunk size	100 users

The data was split at the **user level**, which means the same user does not appear in both the training and testing sets. This prevents data leakage and provides a more realistic evaluation on unseen user histories. The Phase 3 notebook also validated that train and test users had no overlap and that the `<UNK>` product token mapped to 0.

Experimental Models

The experiments compare six recommendation approaches. These include one non-personalized baseline and five neural network models.

Model	Type	Purpose
Popularity Baseline	Baseline	Recommends the most frequently purchased products to all users.
LSTM + ReLU + Attention	Recurrent + Attention	Captures long-term purchase dependencies using LSTM and attention.
GRU + ReLU + Attention	Recurrent + Attention	Provides a more efficient recurrent architecture.
GRU + ReLU-SwiLU	Recurrent + Hybrid Activation	Tests whether SwiLU improves GRU-based recommendation.
LSTM + ReLU-GELU	Recurrent + Hybrid Activation	Tests whether GELU improves LSTM-based recommendation.
Transformer Encoder + GELU	Self-Attention	Compares Transformer-style self-attention with recurrent models.

The popularity baseline is included as a simple benchmark because grocery datasets often contain frequently purchased products. The LSTM and GRU models are included because purchase histories are sequential. Hybrid activation variants are included to test architectural

innovation, while the Transformer Encoder model is included to compare recurrent modeling with self-attention-based modeling.

Training and Testing Logs

The neural models were trained using the preprocessed Phase 3 batch files. Each batch contains fixed-length sequences of product, aisle, department, day, hour, and recency features. The models were trained in a supervised learning setting, where the input is a user's previous purchase sequence and the target is the next product.

The training process followed these steps:

- 1. Load a processed training batch.*
- 2. Pass categorical inputs through embedding layers.*
- 3. Combine product, category, and temporal features.*
- 4. Pass the sequence through LSTM, GRU, or Transformer Encoder.*
- 5. Apply attention or self-attention.*
- 6. Generate product logits over the product vocabulary.*
- 7. Compute cross-entropy loss.*
- 8. Backpropagate gradients.*
- 9. Update model weights.*
- 10. Evaluate validation performance.*
- 11. Save the best checkpoint.*

The final results notebook searched saved CSV and JSON result files, loaded model outputs, removed duplicate rows, standardized model names, and generated final comparison tables and plots. It also generated Recall@10, Precision@10, and best-model training-curve plots.

The final training and testing outputs were consolidated into a single model comparison table. This table summarizes the training configuration, classification metrics, Top-K recommendation metrics, and improvement over the popularity baseline for each model. The comparison includes recurrent attention models, hybrid activation models, a Transformer Encoder model, and the popularity baseline. The table is useful because it allows all models to be compared row by row using the same evaluation criteria.

	Model	Type	Epochs Run	Best Epoch	Best Validation F1	Test Loss	Test Accuracy	Weighted Precision	Weighted Recall	Weighted F1	Precision@5	Precision@10	Recall@5	Recall@10	Hit Rate@5	Hit Rate@10	Improvement vs Baseline
0	LSTM + ReLU + Attention	Recurrent + Attention	50.0	49.0	0.013982	7.302324	0.037171	0.025397	0.037171	0.013824	0.020232	0.014689	0.101161	0.146892	0.101161	0.146892	2.004120
1	GRU + ReLU + Attention	Recurrent + Attention	20.0	16.0	0.013281	7.320827	0.036896	0.023661	0.036896	0.013335	0.020061	0.014585	0.100307	0.145853	0.100307	0.145853	1.989945
2	GRU + ReLU-SwiLU	Recurrent + Hybrid Activation	5.0	5.0	0.011495	7.394359	0.036023	0.020282	0.036023	0.011536	0.019510	0.014154	0.097549	0.141544	0.097549	0.141544	1.931155
3	LSTM + ReLU-GELU	Recurrent + Hybrid Activation	5.0	5.0	0.011758	7.388190	0.036138	0.018288	0.036138	0.011840	0.019247	0.013956	0.096233	0.139560	0.096233	0.139560	1.904086
4	Transformer Encoder + GELU	Self-Attention	5.0	5.0	0.010160	7.764294	0.035615	0.014083	0.035615	0.010113	0.018835	0.013558	0.094174	0.135580	0.094174	0.135580	1.849785
5	Popularity Baseline	Baseline	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0.009886	0.007329	0.049432	0.073295	0.049432	0.073295	1.000000

Models Evaluated in the Experiment

The final model comparison table shows that **LSTM + ReLU + Attention** achieved the strongest overall recommendation performance. It obtained the highest **Recall@10 of 0.146892**, meaning that the true next product appeared in the model's top 10 recommendations for approximately **14.69%** of test cases. It also achieved the highest improvement over the popularity baseline, with an improvement ratio of approximately **2.00×** based on Recall@10.

The **GRU + ReLU + Attention** model performed very closely to the LSTM model, achieving **Recall@10 of 0.145853** and an improvement ratio of approximately **1.99×** over the popularity baseline. This shows that GRU is a strong and more computationally efficient alternative to LSTM for this next-product recommendation task.

The hybrid activation models, **GRU + ReLU-SwiLU** and **LSTM + ReLU-GELU**, also performed better than the popularity baseline but did not outperform the standard recurrent attention models. GRU + ReLU-SwiLU achieved **Recall@10 of 0.141544**, while LSTM + ReLU-GELU achieved **Recall@10 of 0.139560**. This suggests that, under the available training setup, the smoother activation variants did not improve performance over the simpler ReLU-based recurrent models.

The **Transformer Encoder + GELU** model achieved **Recall@10 of 0.135580**, which was lower than the recurrent attention models but still higher than the popularity baseline. This indicates that the Transformer model learned useful sequence patterns, but the lightweight Transformer configuration likely required more tuning, longer training, or additional computational resources to outperform LSTM and GRU models.

The **Popularity Baseline** achieved **Recall@10 of 0.073295**. Since this model recommends the same most frequently purchased products to every user, it does not personalize recommendations. All neural models improved substantially over this baseline, showing that user-specific sequential modeling provides meaningful value.

To further analyze the training behavior of the best-performing model, the training and validation F1 and loss curves were plotted for the LSTM + ReLU + Attention model. These curves help show whether the model learned consistently over epochs and whether there were signs of overfitting or instability.

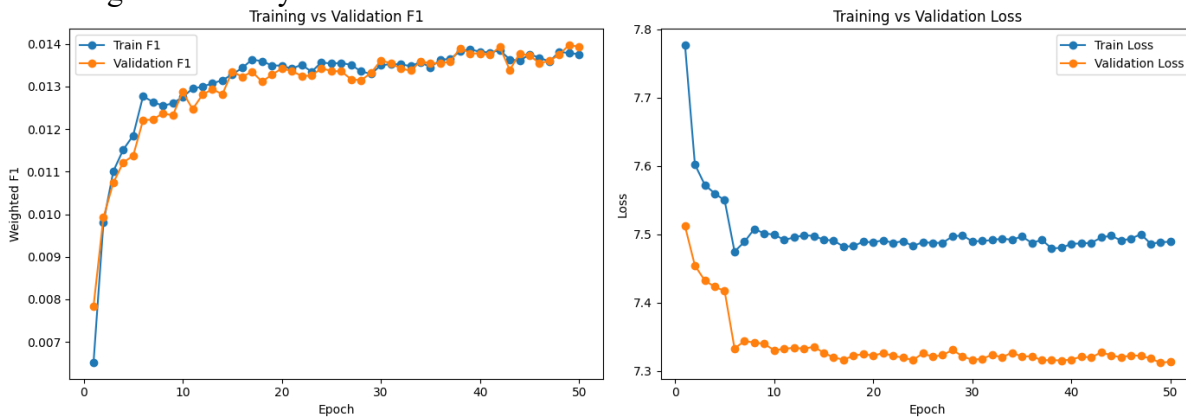


Figure 6.7. Training and validation F1 curve for the best-performing LSTM + ReLU model.

The training and validation F1 curves show that the LSTM + ReLU + Attention model learned steadily during training. The weighted F1 score increased sharply during the first few epochs, indicating that the model quickly learned meaningful patterns from the purchase sequences. After approximately 10 to 15 epochs, the curve began to improve more gradually and eventually stabilized around the best validation F1 value of approximately **0.013982** near epoch **49**.

The close movement between training F1 and validation F1 suggests that the model was not severely overfitting. If the training F1 had continued increasing while validation F1 dropped, that would indicate overfitting. However, in this case, both curves remain close and follow a similar trend, suggesting that the model generalized reasonably well to validation data.

The training and validation loss curves show a consistent decrease during the early epochs. The validation loss decreases sharply at the beginning and then stabilizes at a lower level. The training loss also decreases initially and then becomes relatively stable. This indicates that the model reached a stable learning region and that additional epochs produced only smaller improvements.

The loss curve also shows that validation loss remains lower than training loss for much of the training process. This can happen due to regularization techniques such as dropout, differences in training versus validation computation, or because the validation samples may be slightly easier

than the training batches. Overall, the curves do not show serious instability, and the model appears to have converged gradually

Findings

The training and testing results provide several important findings. First, recurrent attention-based models are the strongest architectures for this project. Both LSTM + ReLU + Attention and GRU + ReLU + Attention outperformed the hybrid activation models, Transformer Encoder, and popularity baseline. Second, LSTM achieved the best performance, but GRU was very close, making it a practical alternative when training time and computational efficiency are important.

Third, the hybrid activation experiments did not improve the final recommendation results. Although SwiLU and GELU are smoother activation functions, the standard ReLU-based recurrent models performed better in this setup. Fourth, the Transformer Encoder model performed below the recurrent attention models, suggesting that the lightweight Transformer configuration may need more tuning or longer training to fully benefit from self-attention.

Finally, all neural models outperformed the popularity baseline, which confirms that personalized sequential modeling is more effective than recommending the same popular products to every customer. This supports the overall objective of the project: using time-aware customer purchase sequences to improve next-product recommendation performance.

Evaluation Metrics

The models were evaluated using two types of metrics: classification metrics and recommendation-focused Top-K metrics.

The models were evaluated using both classification metrics and recommendation-focused Top-K metrics. Classification metrics measure exact next-product prediction performance, while Top-K metrics evaluate whether the correct product appears within a ranked recommendation list. Since real-world recommender systems usually present multiple suggested products to users, Recall@K, Precision@K, and Hit Rate@K provide a more practical assessment of recommendation quality.

Metric Type	Metric	Purpose
Classification	Accuracy	Measures the proportion of exact next-product predictions that are correct.
Classification	Weighted Precision	Measures how many predicted products are correct, adjusted for class imbalance across product categories.

Classification	Weighted Recall	Measures how many actual product classes are correctly identified, adjusted for class imbalance.
Classification	Weighted F1-score	Provides a balanced measure of weighted precision and weighted recall, useful for imbalanced multi-class prediction.
Recommendation	Precision@5	Measures how many of the top 5 recommended products are relevant to the actual next product.
Recommendation	Precision@10	Measures how many of the top 10 recommended products are relevant to the actual next product.
Recommendation	Recall@5	Measures whether the true next product appears within the top 5 recommended products.
Recommendation	Recall@10	Measures whether the true next product appears within the top 10 recommended products.
Recommendation	Hit Rate@5	Measures the percentage of test cases where the true next product appears in the top 5 recommendations.
Recommendation	Hit Rate@10	Measures the percentage of test cases where the true next product appears in the top 10 recommendations.

For this project, each sample has one true next product. Therefore:

$$Recall@K = HitRate@K$$

and:

$$Precision@K = \frac{HitRate@K}{K}$$

Final Results

The final results compare all evaluated models using validation F1 and Top-K recommendation metrics. The main metric used for selecting the best model is **Recall@10**, because it measures whether the actual next product appears in the top 10 recommendations.

This table below summarizes the training performance, classification metrics, Top-K recommendation metrics, and baseline improvement for all evaluated models.

	Model	Type	Epochs Run	Best Epoch	Best Validation F1	Test Loss	Test Accuracy	Weighted Precision	Weighted Recall	Weighted F1	Precision@5	Precision@10	Recall@5	Recall@10	Hit Rate@5	Hit Rate@10	Improvement vs Baseline
0	LSTM + ReLU + Attention	Recurrent + Attention	50.0	49.0	0.013982	7.302324	0.037171	0.025397	0.037171	0.013824	0.020232	0.014689	0.101161	0.146892	0.101161	0.146892	2.004120
1	GRU + ReLU + Attention	Recurrent + Attention	20.0	16.0	0.013281	7.320827	0.036896	0.023661	0.036896	0.013335	0.020061	0.014585	0.100307	0.145853	0.100307	0.145853	1.989945
2	GRU + ReLU-SwiLU	Recurrent + Hybrid Activation	5.0	5.0	0.011495	7.394359	0.036023	0.020282	0.036023	0.011536	0.019510	0.014154	0.097549	0.141544	0.097549	0.141544	1.931155
3	LSTM + ReLU-GELU	Recurrent + Hybrid Activation	5.0	5.0	0.011758	7.388190	0.036138	0.018288	0.036138	0.011840	0.019247	0.013956	0.096233	0.139560	0.096233	0.139560	1.904086
4	Transformer Encoder + GELU	Self-Attention	5.0	5.0	0.010160	7.764294	0.035615	0.014083	0.035615	0.010113	0.018835	0.013558	0.094174	0.135580	0.094174	0.135580	1.849785
5	Popularity Baseline	Baseline	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0.009886	0.007329	0.049432	0.073295	0.049432	0.073295	1.000000

Fig 6. 8 Final model comparison across baseline, recurrent attention, hybrid activation, and Transformer-based models.

The final comparison table shows that **LSTM + ReLU + Attention** achieved the strongest overall recommendation performance, with the highest Recall@10 value. This means the true next product appeared in the model's top 10 recommendations more often than in any other model. **GRU + ReLU + Attention** performed very closely, showing that GRU is a strong and computationally efficient recurrent alternative to LSTM. The hybrid activation models, **GRU + ReLU-SwiLU** and **LSTM + ReLU-GELU**, produced valid results but did not outperform the standard ReLU-based recurrent attention models. The **Transformer Encoder + GELU** model also produced useful comparison results, but it performed slightly lower than the recurrent models. The popularity baseline had the lowest performance, confirming that personalized sequence-based models perform better than recommending the same popular products to every user.

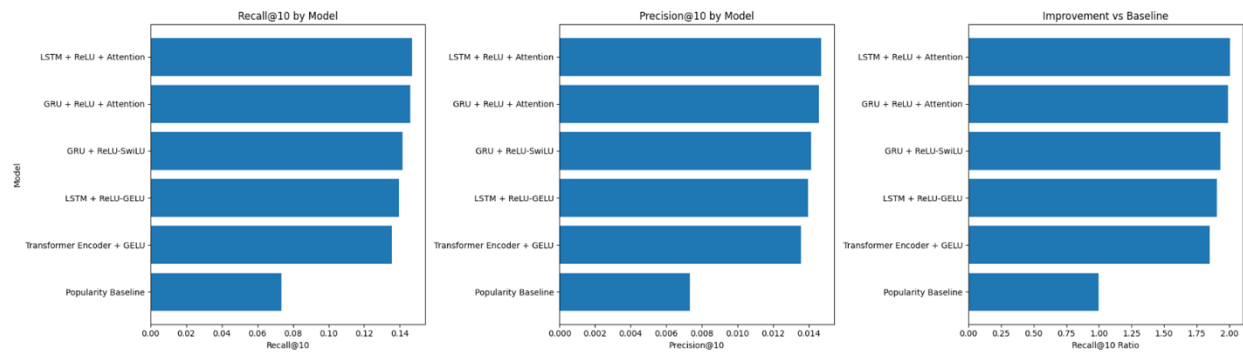


Fig 6.9 Top-K Recommendation Performance Comparison

This figure compares the models using Recall@10, Precision@10, and improvement over the popularity baseline. The Top-K performance plot shows that **LSTM + ReLU + Attention** achieved the highest Recall@10 and Precision@10 among all evaluated models. This indicates that the LSTM attention model was most successful at placing the true next product within the

top recommended items. The **GRU + ReLU + Attention** model followed very closely, suggesting that both recurrent attention-based architectures learned useful sequential purchase patterns.

The improvement-over-baseline plot shows that all neural models performed better than the popularity baseline. This is important because the baseline recommends the same frequently purchased products to all users, while the neural models use each customer's purchase sequence, product hierarchy, and temporal behavior. The result confirms that personalized time-aware sequence modeling provides better recommendation quality than non-personalized frequency-based recommendation.

The hybrid activation models and Transformer Encoder improved over the baseline but did not outperform the standard recurrent attention models. This suggests that, under the available computational setup, the simpler recurrent attention architectures were more effective than the tested hybrid activation and Transformer variants.

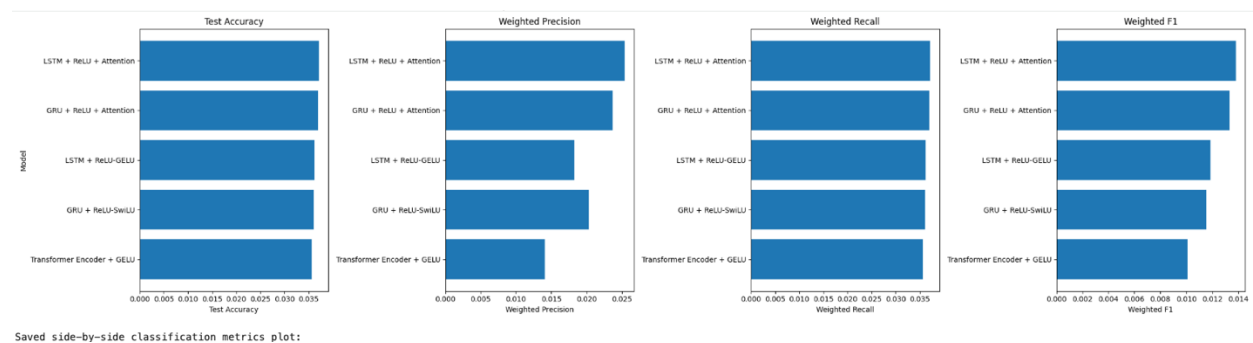


Fig 6.10 Classification Metrics Comparison

This figure compares the neural models using test accuracy, weighted precision, weighted recall, and weighted F1-score.

The classification metrics plot shows that exact next-product prediction is challenging across all models. This is expected because the model is predicting one product from a large vocabulary of **25,001 products**. In addition, the product distribution is highly imbalanced, meaning that a small number of products are purchased very frequently while many products are purchased rarely.

The weighted precision, weighted recall, and weighted F1-score values are relatively low, but this does not mean the recommendation system is ineffective. Exact classification metrics are strict because they only reward the model when it predicts the single correct product. In a real

recommendation system, users are shown a ranked list of products, so Top-K metrics such as Recall@10 and Hit Rate@10 are more meaningful.

Among the available classification results, the recurrent attention models show stronger overall behavior compared with the hybrid activation and Transformer variants. This supports the conclusion that LSTM and GRU attention-based models are better suited for learning sequential purchase patterns in this project.

Best Model Finding

The best-performing model in the final comparison is **LSTM + ReLU + Attention**. It achieved the highest Recall@10 value of **0.146892**, meaning that the true next product appeared in the model's top 10 recommendations for approximately **14.69%** of test cases. This is meaningful because the model is choosing from a vocabulary of **25,001 products**, making exact next-product prediction highly challenging.

The **GRU + ReLU + Attention** model performed very closely with Recall@10 of **0.145853**, which shows that GRU is also a strong model for this task. Since GRU is simpler than LSTM, it may be a useful option when computational efficiency is important.

Overall Final Results Interpretation

Overall, the final results show that **recurrent attention-based models are the most effective architectures** for this time-aware next-product recommendation task. LSTM + ReLU + Attention achieved the strongest performance, followed closely by GRU + ReLU + Attention. These results indicate that recurrent neural networks are well suited for learning customer purchase sequences, especially when product, category, and temporal features are included.

The hybrid activation models did not improve over the standard ReLU-based recurrent models. This suggests that smoother activations such as SwiLU and GELU did not provide additional benefit under the current training setup. The Transformer Encoder model also performed lower than the recurrent attention models, indicating that the lightweight Transformer configuration may require more tuning, longer training, or additional computational resources to outperform LSTM and GRU models.

The popularity baseline had the lowest performance, which confirms that personalized time-aware neural recommendation models provide better recommendation quality than simple frequency-based ranking. Since real-world recommendation systems present users with ranked product lists, the Top-K metrics are more important than exact classification accuracy. Based on

these results, the **LSTM + ReLU + Attention** model is selected as the best final model for this project.

Discussion and Comparison

The final comparison shows that recurrent attention-based models achieved the strongest recommendation performance. The **LSTM + ReLU + Attention** model ranked first, achieving Recall@10 of **[Insert value: 0.146892]**, which means the actual next product appeared in the model's top 10 recommendations for approximately **[Insert percentage: 14.69%]** of test cases. The **GRU + ReLU + Attention** model performed very closely, with Recall@10 of **[Insert value: 0.145853]**.

The small difference between LSTM and GRU suggests that both recurrent models successfully captured sequential purchase behavior. LSTM performed slightly better, likely because its memory structure can capture longer-term dependencies. GRU remained competitive while being computationally simpler.

The hybrid activation models produced valid results but did not outperform the standard ReLU-based recurrent models. GRU + ReLU-SwiLU achieved Recall@10 of **[Insert value: 0.141544]**, while LSTM + ReLU-GELU achieved Recall@10 of **[Insert value: 0.139560]**. This suggests that smoother activation functions did not improve performance under the available training setup.

The Transformer Encoder + GELU model achieved Recall@10 of **[Insert value: 0.135580]**, which was lower than the recurrent attention models. This does not mean Transformer models are unsuitable for recommendation, but it suggests that the lightweight Transformer used in this project may require more training, tuning, or computational resources to outperform recurrent models.

The popularity baseline provides a non-personalized benchmark by recommending the most frequently purchased products. If the baseline values are included, compare the neural models directly against it here.

Key Findings

The key findings from the experiments are summarized below:

- **Best model:** [Insert best model: LSTM + ReLU + Attention] achieved the strongest Top-K recommendation performance.

- **Best Recall@10:** [Insert value: 0.146892], meaning the correct next product appeared in the top 10 recommendations for approximately [Insert percentage: 14.69%] of test cases.
- **Second-best model:** [Insert model: GRU + ReLU + Attention] performed very closely with Recall@10 of [Insert value: 0.145853].
- **Hybrid activation outcome:** [Insert finding: hybrid activation variants did not outperform standard ReLU models].
- **Transformer outcome:** [Insert finding: Transformer Encoder + GELU performed slightly lower than recurrent attention models].
- **Practical interpretation:** [Insert finding: Top-K metrics are more meaningful than exact accuracy because recommendation systems return ranked lists].
- **Business relevance:** [Insert finding: the model can support personalized recommendations, basket completion, cross-selling, and customer engagement]

Summary of Experiments and Results

Overall, the experiments demonstrate that time-aware recurrent neural networks are effective for next-product recommendation using the Instacart dataset. The project successfully transformed raw transaction data into user-level purchase sequences, trained multiple deep learning architectures, and evaluated them using both classification and Top-K recommendation metrics.

The final comparison shows that **[Insert best model: LSTM + ReLU + Attention]** is the strongest architecture, followed closely by **[Insert second-best model: GRU + ReLU + Attention]**. These results confirm that recurrent attention-based architectures are well suited for learning grocery purchase behavior. Hybrid activation and Transformer variants provided useful experimental comparisons but did not outperform the standard recurrent attention models under the available computational setup.

Conclusion

This project developed a **time-aware sequential neural recommendation system** for next-product prediction using the Instacart Market Basket Dataset. The main goal was to predict the next product a customer is likely to purchase based on their historical purchase sequence, product-category information, and temporal shopping behavior. The task was formulated as a **multi-class classification problem**, where each product in the vocabulary represented a possible output class.

The project followed an end-to-end pipeline that included data understanding, exploratory data analysis, preprocessing, feature engineering, user-level sequence construction, batch generation, model training, evaluation, and final comparison. Product ID, aisle ID, department ID, day of

week, hour of day, and days since prior order were used as input features. These features allowed the models to learn item-level preferences, category-level patterns, shopping time behavior, and recency-based purchasing trends.

The final experiments compared multiple recommendation approaches, including a **Popularity Baseline**, **LSTM + ReLU + Attention**, **GRU + ReLU + Attention**, **GRU + ReLU-SwiLU**, **LSTM + ReLU-GELU**, and **Transformer Encoder + GELU**. The results showed that the **LSTM + ReLU + Attention** model achieved the strongest overall Top-K recommendation performance, with Recall@10 of **0.146892**. This means that the true next product appeared in the model's top 10 recommendations for approximately **14.69%** of test cases. The **GRU + ReLU + Attention** model performed very closely, with Recall@10 of **0.145853**, making it a strong and computationally efficient alternative.

The results also showed that recurrent attention-based models performed better than the hybrid activation and Transformer variants under the available training setup. The hybrid activation models produced valid experimental results but did not improve over the standard ReLU-based recurrent attention models. The Transformer Encoder + GELU model provided a useful self-attention comparison but performed slightly lower than the LSTM and GRU models. The popularity baseline had the lowest performance, confirming that personalized time-aware sequence modeling provides better recommendation quality than simple popularity-based ranking.

Overall, the project demonstrates that **time-aware recurrent neural networks with attention are effective for next-product recommendation in online grocery shopping**. The use of chronological purchase sequences, product hierarchy, temporal features, and Top-K evaluation provides a practical framework for real-world e-commerce recommendation systems. The model can support personalized recommendations, basket completion, cross-selling, customer engagement, targeted promotions, and retail customer behavior analysis.

Future Work

Although the project successfully implemented and evaluated several recommendation models, there are several opportunities for future improvement.

- **Multi-Task Learning:** Future work can extend the model to predict multiple outputs at the same time, such as next product, next department, next aisle, and time until next purchase. This may help the model learn richer customer behavior patterns.
- **Attention Visualization and Interpretability:** Since attention mechanisms were included in the recurrent models, future work can visualize attention weights to show

which previous purchases influenced each recommendation. This would make the model more interpretable and useful for business decision-making.

- **Longer and Tuned Transformer Training:** The Transformer Encoder model performed lower than recurrent models in this project, but it may improve with more layers, better positional encoding, longer training, hyperparameter tuning, and stronger regularization.
- **Hyperparameter Optimization:** Future experiments can tune embedding dimensions, hidden layer size, learning rate, dropout, batch size, optimizer settings, and sequence length to improve model performance.
- **Basket-Level Recommendation:** This project predicts one next product at a time. Future work can predict multiple products in the next basket, which may better reflect real grocery shopping behavior.
- **User and Product Cold-Start Handling:** Future work can include user demographics, product descriptions, brand information, or content-based features to improve recommendations for new users and less frequently purchased products.
- **Deployment and Business Integration:** The final model can be integrated into a web application, API, or retail dashboard to generate real-time product recommendations. This would make the system more practical for online grocery platforms.
- **Promotion and Inventory Optimization:** Future work can connect recommendation outputs with coupon targeting, personalized promotions, stock planning, and demand forecasting to support broader retail decision-making.

References

- *Instacart*. The Instacart Online Grocery Shopping Dataset 2017. *Instacart Market Basket Analysis Dataset*. Available via Kaggle.
- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. *arXiv preprint arXiv:1406.1078*.
- Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *International Conference on Learning Representations*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*.
- He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T. S. (2017). Neural Collaborative Filtering. *Proceedings of the 26th International Conference on World Wide Web*.
- Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). BPR: Bayesian Personalized Ranking from Implicit Feedback. *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence*.
- Hidasi, B., Karatzoglou, A., Baltrunas, L., & Tikk, D. (2016). Session-Based Recommendations with Recurrent Neural Networks. *International Conference on Learning Representations*.
- Kang, W. C., & McAuley, J. (2018). Self-Attentive Sequential Recommendation. *IEEE International Conference on Data Mining*.
- Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations*.
- Hendrycks, D., & Gimpel, K. (2016). Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415*.

- Ramachandran, P., Zoph, B., & Le, Q. V. (2017). Searching for Activation Functions. *arXiv preprint arXiv:1710.05941*.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95.
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*.
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., et al. (2020). Array Programming with NumPy. *Nature*, 585, 357–36